VERIQTA

PRODUCTION
KNOWLEDGE
REAL SYSTEMS
BETTER ENGINEERS

SENIOR NETWORKING BIBLE FOR DEVOPS

ARCHITECTURE | TROUBLESHOOTING | SECURITY | SCALING
PERFORMANCE | OBSERVABILITY | REAL-WORLD SCENARIOS



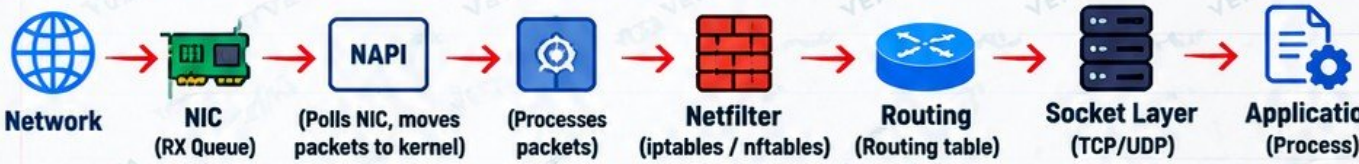
@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

The Linux Packet Path: From NIC to Application

Trace ingress and egress through NIC queues, NAPI, softirq, routing, netfilter, sockets, and network namespacas. Diagnose packet drops, CPU saturation, and kernel bottlenecks.

Production Case: Packets arrive at the host, but the application never receives them.

1. INGRESS PACKET FLOW (NIC TO APPLICATION)



2. EGRESS PACKET FLOW (APPLICATION TO NIC)



KEY KERNEL COMPONENTS

1	**NIC / Driver**	Receives packets into RX queues using DMA.
2	**NAPI**	Polls RX queues, reduces interrupts.
3	**SoftIRQ**	Processes packets in kernel context.
4	**Netfilter**	iptables/nftables for filtering, NAT and firewall.
5	**Routing**	Route lookup and destination decision.
6	**Socket Layer**	Delivers packets to the correct process (TCP/UDP).
7	**Network Namespaces**	Isolates networking for containers and processes.

COMMON PACKET DROP POINTS

1	**NIC RX queue overflow**	High traffic or insufficient IRQ handling.
2	**SoftIRQ / CPU saturation**	Kernel cannot process packets fast enough.
3	**Netfilter rules**	iptables/nftables dropping packets.
4	**Routing issues**	No route or incorrect route.
5	**Connection tracking**	Conntrack table full (NAT/firewall).
6	**Socket buffer full**	Application not reading fast enough.
7	**Network namespace**	Packet in wrong namespace.

USEFUL DIAGNOSTIC COMMANDS

```
1  ip link show              # Check interface state and drops
2  ethtool -S eth0           # NIC driver statistics
3  nstat -az                 # Network stack statistics
3  ss -s                     # Socket summary
4  ss -antp                  # Active TCP connections
5  ip route show             # Routing table
6  iptables -L -v -n         # Check iptables rules and counters
7  nft list ruleset          # Check nftables rules
7  conntrack -S              # Connection tracking statistics
#  ip netns list             # List network namespaces
#  tcpdump -i eth0 -nn       # Capture packets on interface
```

TROUBLESHOOTING APPROACH

1. **Confirm packets arrive at the NIC**
   Check interface counters with `ip -s link` and `ethtool -S eth0`.
2. **Check for drops before netfilter**
   Look at NIC, kernel and SoftIRQ statistics (`nstat`, `dmesg`).
3. **Validate firewall and routing**
   Check iptables/nftables counters and routing table.
4. **Check connection tracking**
   Look for table saturation with `conntrack -S`.
5. **Verify delivery to socket**
   Use `ss -antp` to see if the application has a listening socket.
6. **Check application and namespace**
   Confirm process is running in the correct network namespace and reading from the socket.

SENIOR ENGINEER INSIGHT

- Packet drops can occur long before iptables or the application.
- High SoftIRQ CPU usage is a common cause of unexplained packet loss.
- Containers add another layer of isolation through network namespaces.
- Always correlate interface, kernel and application metrics.

REAL-WORLD TIP

- Monitor NIC drops, SoftIRQ CPU, conntrack usage and socket buffer utilization.
- Use eBPF tools (e.g. **bpftool**, **tc**, **tracepoints**) for deep packet path visibility.
- Check for asymmetric routing and MTU issues if packets are seen but connections fail.

COMMON MISTAKE

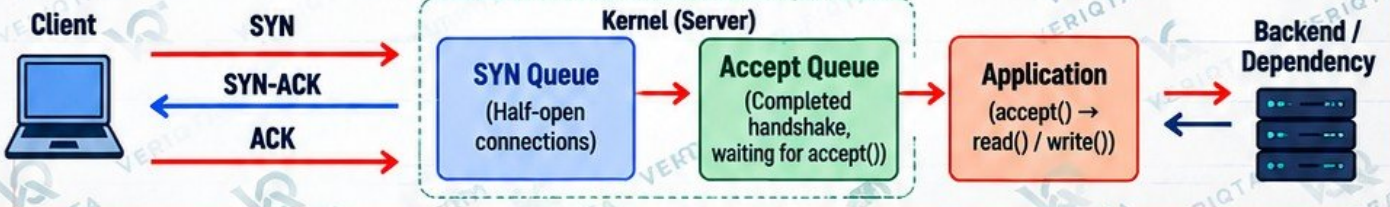
- Focusing only on the application.
- Ignoring NIC and kernel counters.
- Assuming packets are dropped only by iptables.
- Not checking network namespaces in container environments.

TCP Under Pressure: Queues, Retransmissions and Collapse

Examine SYN and accept queues, congestion control, receive windows, retransmission timeouts, and connection lifecycle. Separate network congestion from overloaded applications.

Production Case: P99 latency rises while CPU and memory appear healthy.

1. TCP CONNECTION LIFECYCLE AND QUEUES



2. KEY QUEUES AND WHAT THEY MEAN

Queue	What it Holds	Key Limits	Common Issue
SYN queue	Half-open connections (SYN received)	tcp_max_syn_backlog	Full queue → dropped SYN packets
Accept queue	Completed handshakes (waiting for accept())	somaxconn (backlog)	Full queue → clients retry or timeout
Receive queue	Data received from client	rmem settings	Full queue → window closes → client slows
Send queue	Data waiting to be sent	wmem settings	Full queue → higher latency or timeouts

3. TCP CONGESTION CONTROL

- Controls how much data is sent based on network conditions.
- Increases congestion window (cwnd) when no loss.
- Reduces cwnd on packet loss or high delay.
- Common algorithms: CUBIC (Linux default), BBR.
- Wrong algorithm or misconfigured parameters can increase latency or cause throughput collapse.



4. RETRANSMISSIONS AND TIMEOUTS

- If an ACK is not received, TCP retransmits the packet.
- Retransmission timeout (RTO) is calculated dynamically based on RTT (Round Trip Time).
- High latency, packet loss, or bufferbloat increases RTO and retransmissions, which reduces throughput and increases P99.
- Excessive retransmissions indicate network congestion or downstream issues.

5. COMMON SYMPTOMS AND LIKELY CAUSES

Symptom	Likely Cause	Where to Check
High P99 latency	Network congestion, retransmissions, queueing	ss, nstat, tcpdump
Connection timeouts	Full SYN/accept queue, firewall, packet loss	ss, netstat, dmesg
Low throughput	Congestion control, high RTT, packet loss	nstat, tc, metrics
Many SYN_RECV	SYN flood or high latency	ss, netstat
Many TIME_WAIT	High connection churn	ss, nf_conntrack

6. USEFUL DIAGNOSTIC COMMANDS

```
1  ss -s                          # TCP summary (queues, states)
2  ss -tln                        # Listening sockets and queues
3  ss -nto state syn-recv         # Check SYN queue
4  netstat -s | grep -i tcp       # TCP statistics
5  nstat -az                      # Detailed TCP metrics
6  tcpdump -nn -i eth0            # Packet capture
7  dmesg | grep -i tcp            # Kernel TCP messages
8  cat /proc/sys/net/ipv4/tcp_*   # TCP settings
9  ss -nti                        # Per-connection TCP details
10 tc -s qdisc                    # Queueing and drops (if using tc)
```

7. TUNING AND BEST PRACTICES

- Size `somaxconn` and `tcp_max_syn_backlog` appropriately.
- Use modern congestion control (BBR for high-latency links).
- Monitor retransmissions, RTT, and queue lengths.
- Avoid excessive connection churn (use connection pooling).
- Ensure adequate receive and send buffer sizes.
- Use load balancing and multiple instances to spread load.
- Identify network versus application issues using packet captures, TCP metrics and queue statistics.

SENIOR ENGINEER INSIGHT

- Increasing somaxconn alone does not fix latency if the application cannot accept connections fast enough.
- High retransmissions with low CPU usually indicates network congestion or downstream latency.
- Use evidence before tuning - measure queues, RTT, retransmissions and drops.

REAL-WORLD TIP

- Compare client-side and server-side metrics. If retransmissions are high on both sides, it's a network issue. If only the server side shows high queues and accept delays, it's an application bottleneck.

COMMON MISTAKE

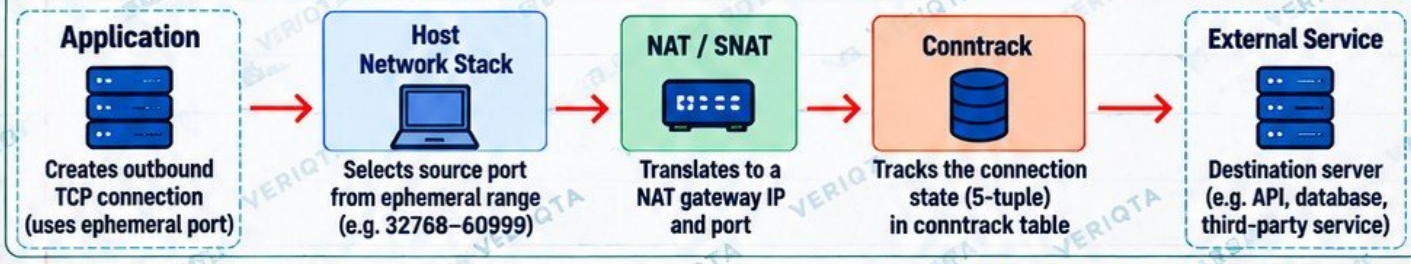
- Always assuming high latency is an application problem.
- Ignoring retransmissions and queue statistics.
- Changing multiple kernel settings without measuring the impact.

Connection Exhaustion: Sockets, NAT and Conntrack

Investigate ephemeral port exhaustion, TIME_WAIT, file descriptors, connection tracking, SNAT port allocation, and connection pooling. Explain why increasing limits can move rather than solve the bottleneck.

Production Case: Outbound connections intermittently fail during traffic spikes.

1. OUTBOUND CONNECTION FLOW (WITH NAT AND CONNTRACK)



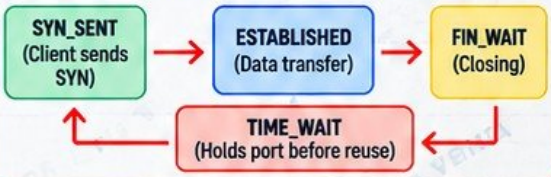
2. KEY COMPONENTS AND LIMITS

Component	Purpose	Typical Limit / Default	What Happens When Exhausted
Ephemeral Ports	Source ports for outbound connections	~ 32768 – 60999 (28K ports)	New connections fail (e.g. EADDRNOTAVAIL).
TIME_WAIT	Holds closed connections for a period	60 seconds (default)	Ports remain unavailable temporarily.
File Descriptors	Limits number of open sockets per process	ulimit -n (usually 1024)	Application cannot open new connections.
Conntrack Table	Tracks NAT connections	nf_conntrack_max (default varies)	New connections dropped when table is full.
SNAT Port Allocation	NAT gateway translates multiple hosts to one IP	Limited by available ports per NAT IP	Port exhaustion causes intermittent connection failures.

3. COMMON FAILURE SYMPTOMS

- Outbound connections randomly fail.
- Errors like EADDRNOTAVAIL, connection timed out or no route to host.
- High TIME_WAIT count.
- Conntrack table full (nf_conntrack: table full).
- Spikes in connection creation during peak traffic.
- Works again after some time (ports freed).
- Application and server appear healthy.

4. COMNECTION STATES (SIMPLIFIED)



5. USEFUL DIAGNOSTIC COMMANDS

```
1  ss -s                                  # Socket summary (established, timewait, etc.)
2  ss -antp | grep TIME-WAIT |wc -l       # Count TIME_WAIT connections
3  ss -antp | grep :443                   # Check connections to a specific port
4  cat /proc/sys/net/ipv4/ip_local_port_range   # View ephemeral port range
5  cat /proc/sys/net/ipv4/tcp_fin_timeout       # View TIME_WAIT duration
6  lsof -p <pid> | wc -l                  # Check open file descriptors for a process
7  cat /proc/sys/net/nf_conntrack_max     # Check conntrack table size
8  cat /proc/net/nf_conntrack | wc -l     # Current conntrack entries
9  dmesg | grep -i conntrack              # Look for conntrack related errors
10 netstat -s | grep -iE "listen|fail|drop"   # Check network stack statistics
```

6. TUNING EXAMPLES (USE WITH CARE)

```
# Increase ephemeral port range
sysctl -w net.ipv4.ip_local_port_range="10240 65535"

# Reduce TIME_WAIT duration (use carefully)
sysctl -w net.ipv4.tcp_fin_timeout=30

# Increase file descriptor limit for a process
ulimit -n 65536

# Increase conntrack table size
sysctl -w net.netfilter.nf_conntrack_max=262144

# Enable connection reuse in application
Use HTTP keep-alive / connection pooling
(e.g. in Nginx, Java, Python, Go libraries)
```

SENIOR ENGINEER INSIGHT

- Increasing port range or conntrack size only moves the bottleneck if connection churn remains high.
- Use connection pooling and keep-alive to reduce new connection creation.
- Investigate why connections are not being reused (e.g. application settings, load balancer, idle timeouts).
- Monitor conntrack usage, TIME_WAIT, and new connection rate during peak traffic.

8. COMMON ROOT CAUSES

- High connection churn (no connection pooling).
- Short-lived HTTP clients.
- Misconfigured load balancer or proxy (closing connections).
- Insufficient SNAT ports on NAT gateway.
- Conntrack table too small for traffic volume.
- Third-party service latency causing increased concurrent connections.

9. BEST PRACTICES

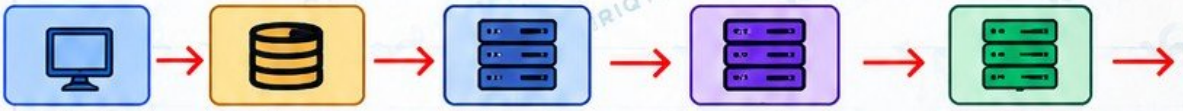
- Use connection pooling and reuse.
- Set appropriate keep-alive and idle timeouts.
- Monitor ephemeral port, TIME_WAIT and conntrack metrics.
- Right-size NAT gateways or use NAT instance with more ports if needed.
- Use multiple NAT IPs for high outbound traffic.
- Investigate and reduce unnecessary retries.

DNS Failure Engineering: Caches, TTLs and Resolver Chains

Follow resolution through application caches, stub resolvers, recursive resolvers, authoritative servers, and Kubernetes DNS. Examine negative caching, retry amplification, stale records, and query saturation.

Production Case: Service discovery fails intermittently although DNS records are correct.

1. DNS RESOLUTION FLOW (TYPICAL PATH)



Application (Uses local cache if available) → **Application Cache** (e.g. JVM, Go, Node.js, libc) → **Stub Resolver** (/etc/resolv.conf) (e.g. systemd-resolved, glibc, nscd) → **Recursive Resolver** (e.g. CoreDNS, dnsmasq, BIND, Unbound, Amazon Route 53) → **Authoritative Server** (e.g. Route 53, Cloud DNS, Internal DNS) → **Kubernetes DNS (CoreDNS)** (for cluster services)

2. KEY DNS CACHES AND TTL BEHAVIOUR

Layer	What it Caches	TTL Behaviour	Key Considerations
Application Cache	Domain names and responses	Often custom and can ignore low TTL	Can serve stale records. Must respect TTL or support refresh.
Stub Resolver	Cached responses (positive and negative)	Follows TTL. Negative TTL from SOA.	systemd-resolved, glibc, nscd have different behaviours.
Recursive Resolver	Cached responses	Follows record TTL. Negative caching based on SOA record.	High TTL can delay changes. Negative caching can hide new records.
Authoritative Server	Authoritative records	Defines TTL and negative TTL (SOA).	Incorrect TTL or SOA settings can cause long outages.
Kubernetes DNS (CoreDNS)	Cluster service and external records (with caching)	Default 30s (positive) and 5s (negative) (configurable).	Large query volume can cause delays or timeouts.

3. NEGATIVE CACHING AND FAILURE SCENARIOS

Scenario	What Happens	Impact
NXDOMAIN cached	Resolver caches non-existent domain based on SOA negative TTL.	Newly created record not found until TTL expires.
SERVFAIL cached	Temporary DNS failure cached as negative response.	Intermittent failures even after issue is fixed.
Stale records	High TTL or application cache keeps old IPs.	Traffic sent to wrong or terminated instances.
Retry amplification	Clients retry aggressively on timeout or failure.	Massive query spikes and resolver saturation.
Query saturation	High QPS or low TTL from many clients (e.g. microservices).	Increased latency, timeouts, SERVFAIL.

4. COMMON DNS FAILURE SYMPTOMS

- Intermittent name resolution failures (timeout, SERVFAIL).
- Applications can resolve some domains but not others.
- High DNS latency and increased query rate.
- NXDOMAIN for recently created records.
- Kubernetes services not resolving or resolving to old Pods.
- Works after flushing cache or restarting resolver.
- Only fails during traffic spikes.

5. USEFUL DIAGNOSTIC COMMANDS

```
1  dig example.com                          # Full resolution details
2  dig example.com +trace                   # Trace full resolution path
3  dig example.com @8.8.8.8                 # Query specific resolver
4  dig example.com +ttlunits                # Show TTL in human readable
5  dig -x <ip>                              # Reverse DNS lookup
6  nslookup example.com                     # Simple query
7  host example.com                         # Quick lookup
8  resolvectl status                        # systemd-resolved status
9  cat /etc/resolv.conf                     # Check resolver configuration
10 kubectl exec -n kube-system \
     deploy/coredns -- dig example.com      # Test from CoreDNS Pod
```

6. TUNING AND BEST PRACTICES

- Use appropriate TTL values for your environment.
- Avoid excessive client-side caching or ignoring TTL.
- Configure reasonable negative TTL in authoritative DNS.
- Monitor DNS latency, error rate and query volume.
- Implement retries with backoff and jitter (avoid amplification).
- Use local caching resolvers (e.g. CoreDNS, Unbound).
- Ensure high availability for DNS resolvers.
- Validate DNS changes before production (lower TTL, test).
- For Kubernetes, tune CoreDNS cache and consider NodeLocal DNS.

SENIOR ENGINEER INSIGHT

- DNS failures often appear as application or network issues. Always verify DNS early in incident investigation.
- Negative caching and application caches are common causes of "it's fixed but still failing".
- Low TTL is not always better. Very low TTL can cause query saturation and higher latency.
- Correlate DNS metrics with application errors and retries.

REAL-WORLD TIPS

- During incident, check from multiple locations: pod, node, VPC, external network.
- Compare results from different resolvers.
- Check SOA record for negative TTL.
- Look for spikes in DNS query volume and latency.
- Validate that clients respect TTL.
- Use caching resolvers close to workloads.

COMMON MISTAKES

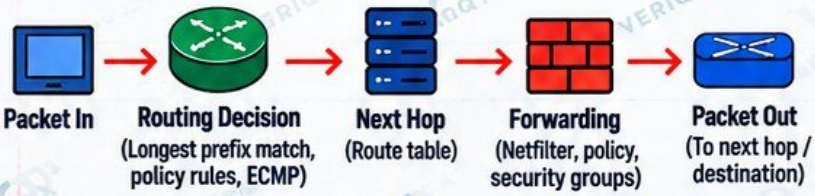
- Assuming DNS is always fast and reliable.
- Ignoring negative caching behaviour.
- Setting very high or very low TTL without understanding the impact.
- Not monitoring DNS query volume and errors.
- Forgetting application-level DNS caches.
- Not testing DNS changes before rollout.

Routing Failures: Asymmetry, MTU and Black Holes

Analyze route selection, policy routing, reverse-path filtering, PMTU discovery, fragmentation, ECMP, and asymmetric return paths.

Production Case: Small requests succeed while larger TLS requests hang.

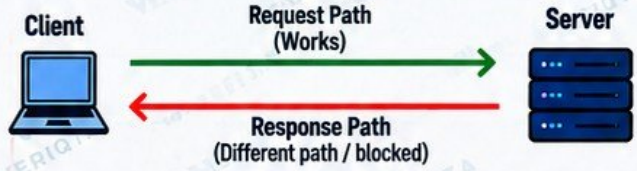
1. HOW ROUTING WORKS (SIMPLIFIED FLOW)



2. COMMON ROUTING FAILURE PATTERNS

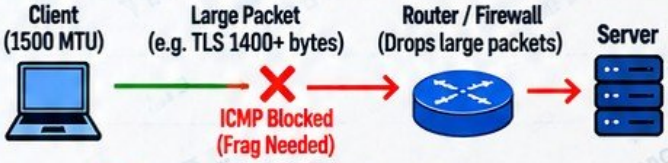
- Asymmetric routing (different path in/out).
- Incorrect or missing route (black hole).
- Reverse-path filtering dropping legitimate traffic.
- MTU issues (packets too large, fragmentation blocked).
- ECMP sending flows over inconsistent paths.
- Policy routing sending traffic to wrong table.
- NAT or firewall blocking return traffic.
- Routes present but next hop unreachable.

3. ASYMMETRIC ROUTING



Asymmetric routing can cause stateful firewalls, reverse-path filtering, or security groups to drop return traffic.

4. MTU, PMTU DISCOVERY AND FRAGMENTATION



- Small packets (e.g. 100 bytes) → Work
- Large packets → Hang (no ICMP response, no fragmentation)

5. KEY CONCEPTS

Concept	What It Means	Key Issues	How to Check / Fix
Route selection	Longest prefix match, metrics, policy rules	Wrong route or next hop	`ip route, ip rule`
Policy routing	Multiple routing tables based on rules	Traffic sent to wrong table	`ip rule show` `ip route show table <id>`
Reverse-path filtering (RPF)	Validates source IP is reachable via incoming interface	Drops asymmetric traffic	`sysctl net.ipv4.conf.all.rp_filter`
PMTU discovery	Finds the largest packet size on path	ICMP blocked or ignored	`tracepath, ping -M do` Look for "Frag needed"
Fragmentation	Splits packets when too large	Fragments dropped by firewalls	`tcpdump`, check DF bit
ECMP	Multiple equal-cost paths	Inconsistent path, out-of-order packets	`ip route, traceroute` `flow hashing`

6. USEFUL DIAGNOSTIC COMMANDS

```
1  ip route show                          # View routing table
2  ip rule show                           # View policy routing rules
3  ip route get <ip>                      # See selected route
4  traceroute <ip>                        # Check path and asymmetry
5  tracepath <ip>                         # Check MTU along path
6  ping -M do -s <size> <ip>              # Test PMTU (no fragmentation)
7  ping -M do -s 1472 <ip>                # Test common MTU size
8  iptables -L -v -n                      # Check firewall rules
9  tcpdump -i <iface> icmp                # Look for ICMP "Frag needed"
10 sysctl net.ipv4.conf.all.rp_filter     # Check reverse-path filtering
11 ss -tn state established               # Check established connections
12 mtr <ip>                               # Real-time path analysis
```

7. REAL-WORLD SCENARIO ANALYSIS

Small requests succeed, larger TLS requests hang

1. Client sends small packets → works (e.g. 100 bytes).
2. Client sends large TLS packets → hang (e.g. 1400+ bytes).
3. ICMP "Fragmentation Needed" is blocked by firewall.
4. PMTU discovery fails, packets are dropped.
5. No fragmentation allowed (DF bit set).
6. Connection stalls during TLS handshake or data transfer.

8. TROUBLESHOOTING STEPS

1. Identify the exact point of failure (client, server, or network).
2. Test different packet sizes with `ping -M do`.
3. Check if ICMP is blocked on the path.
4. Verify routing tables and next hops.
5. Check for asymmetric routing using `traceroute/mtr`.
6. Validate MTU settings end-to-end.
7. Review firewall/NAT/security group rules.
8. Confirm reverse-path filtering settings.

9. BEST PRACTICES

- Ensure correct and consistent routing for both directions.
- Allow necessary ICMP for PMTU discovery.
- Use consistent MTU across the path.
- Monitor for asymmetric routing.
- Validate routing changes with traceroute and real traffic.
- Document and test policy routing rules.
- Use monitoring to detect black holes and packet drops.
- Always test with realistic packet sizes (including TLS traffic).

SENIOR ENGINEER INSIGHT

- Most "mysterious" network timeouts are MTU or asymmetric routing issues rather than application bugs.
- Blocking ICMP can break PMTU discovery and cause silent failures.
- Always test with the same packet sizes and protocols as production. (e.g. TLS).

REAL-WORLD TIP

- If small packets work but large packets hang, check ICMP, MTU and firewall rules first.
- Use tracepath to quickly identify the MTU on the path.
- Look for asymmetric routing when only one direction is affected.

COMMON MISTAKES

- Assuming routes are symmetric.
- Blocking ICMP without understanding PMTU.
- Ignoring MTU issues in container, VPN or cloud environments.
- Not testing with large packets or TLS.

Load Balancers, Proxies and Hidden Failure Domains

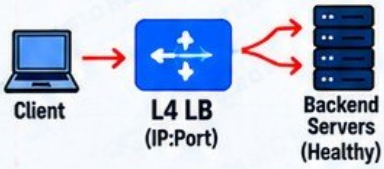
Compare L4 and L7 routing, health checks, connection draining, keep-alive, TLS termination, upstream pools, and fail-open versus fail-closed behavior.

Production Case: Healthy backend instances still produce widespread 502 and 504 errors.

1. L4 vs L7 LOAD BALANCING (KEY DIFFERENCES)

L4 LOAD BALANCER (Transport Layer)

- Routes based on IP address and port.
- Works with TCP/UDP.
- Faster and lower latency.
- Does not inspect application data.
- Common: AWS NLB, TCP load balancers.

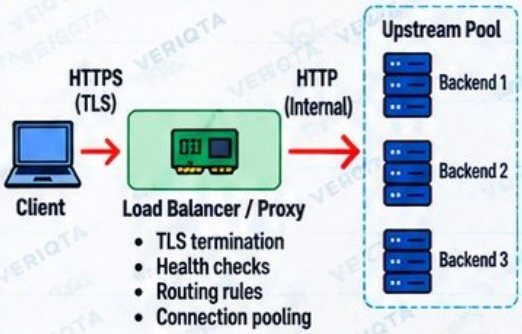


L7 LOAD BALANCER (Application Layer)

- Routes based on HTTP/HTTPS (host, path, headers).
- Can terminate TLS (SSL offload).
- Supports advanced routing and rewrites.
- Performs health checks at application level.
- Common: NGINX, ALB, HAProxy, Envoy, API Gateway.



2. REQUEST FLOW WITH TLS TERMINATION



3. KEY COMPONENTS AND THEIR ROLE

Component	Purpose	Key Configuration	Common Issues
Listener	Listens for incoming traffic (IP:Port, protocol)	Port, protocol (HTTP/HTTPS/TCP)	Wrong protocol, certificate issues
TLS Termination	Decrypts HTTPS at the LB	Certificates, cipher settings	Expired cert, wrong SNI, incompatible ciphers
Health Checks	Checks backend health	Path, port, interval, timeout, success codes	False positives, wrong path, slow responses
Connection Draining	Allows existing connections to finish before removing a backend	Drain timeout	Requests cut off during deploys
Keep-Alive	Reuses TCP connections	Keep-alive timeout, max requests	Too short (high connection churn) or too long (stale connections)
Upstream Pool	Set of backend instances	Load balancing algorithm (round robin, least conn, etc.)	Uneven load, unhealthy instances
Fail-Open / Fail-Closed	Behavior when all backends are unhealthy	Fail-open or fail-closed setting	Fail-open can send traffic to bad instances, fail-closed returns 502/504

4. COMMON 502 AND 504 CAUSES (EVEN WHEN BACKENDS ARE HEALTHY)

Cause	Explanation
Health check mismatch	LB marks instances unhealthy due to wrong path, headers or success codes.
TLS/SNI issues	Wrong certificate, missing intermediate certs, or SNI mismatch.
Connection timeout	LB timeout is lower than backend response time.
Keep-alive / idle timeout	LB closes idle connections while client still reuses them.
SNAT/ephemeral port exhaustion	LB or proxy runs out of source ports for upstream connections.
Load balancer overload	LB reaches connection or request rate limits.
Incorrect routing rules	Requrests routed to wrong backend or default backend.
Network ACL / Security Group	Intermittent drops between LB and backend.
Hidden failure domain	One AZ, subnet or node group partially failing.
Fail-open misconfiguration	Traffic sent to unhealthy or terminating instances.

5. USEFUL DIAGNOSTIC COMMANDS

```
1  curl -v https://example.com                          # Check response and TLS details
2  curl -vk https://example.com                         # Skip certificate verification (test)
3  openssl s_client -connect example.com:443            # Check TLS handshake and cert chain
   -servername example.com
4  dig example.com                                      # DNS resolution
5  ss -tn | grep :443                                   # Active connections to port 443
6  netstat -anp | grep ESTAB                            # Established connections
7  tcpdump -i eth0 port 443                             # Capture TLS traffic
8  kubectl get endpoints <service>                      # Check backend endpoints (K8s)
9  kubectl describe ingress <name>                      # Check ingress configuration
10 kubectl logs <ingress-controller>                    # View ingress controller logs
```

6. HEALTH CHECK EXAMPLE (NGINX)

```
upstream backend_pool {
    server 10.0.1.10:8080 max_fails=3 fail_timeout=30s;
    server 10.0.1.11:8080 max_fails=3 fail_timeout=30s;
    server 10.0.1.12:8080 max_fails=3 fail_timeout=30s;
}
server {
    listen 443 ssl;
    server_name example.com;
    location / {
        proxy_pass http://backend_pool;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header Host $host;
        proxy_connect_timeout 5s;
        proxy_read_timeout 30s;
        proxy_send_timeout 30s;
    }
```

SENIOR ENGINEER INSIGHT

- A healthy instance at the application layer does not mean it is reachable, correctly routed, or passing health checks.
- Most 502/504 issues are caused by timeouts, connection management, or misconfiguration rather than the application itself.
- Always validate the full path: client → LB → network → backend → application.

REAL-WORLD TIP

- Align LB timeouts with application timeouts.
- Use separate health check endpoints.
- Monitor LB metrics: active connections, request rates, 5xx errors, target health.
- Test connection draining during deployments.
- Use multiple instances and multi-AZ to avoid single failure domains.

COMMON MISTAKES

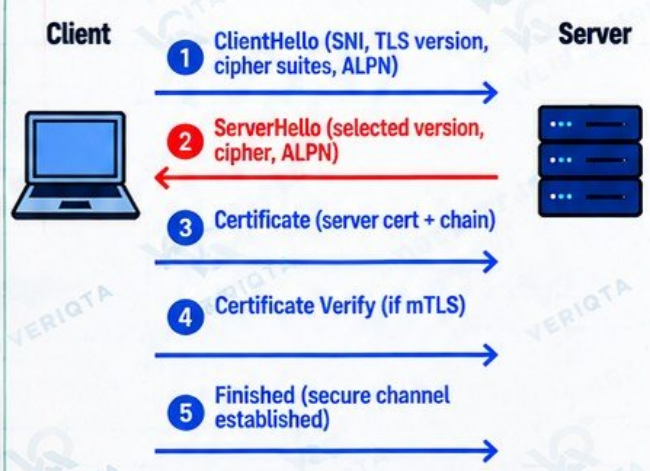
- Using HTTP health checks for HTTPS services without proper configuration.
- Setting LB timeouts too low.
- Ignoring TLS certificate and SNI issues.
- Not testing fail-open / fail-closed behavior.
- Assuming a 200 response from / means the whole application is healthy.

TLS and mTLS: Handshake Failures at Scale

Explore certificate chains, SNI, ALPN, TLS negotiation, mutual authentication, certificate rotation, session resumption, and trust-store divergence.

Production Case: A certificate rotation causes partial service-to-service communication failure.

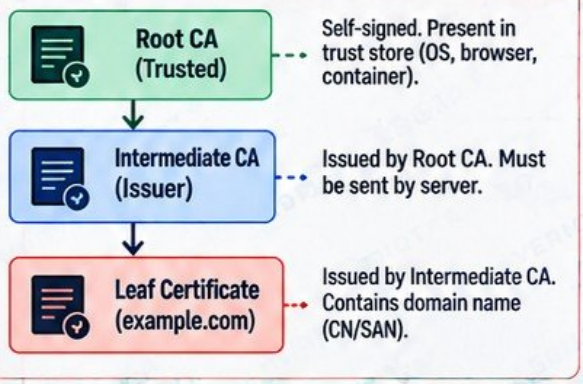
1. TLS HANDSHAKE FLOW (SIMPLIFIED)



2. KEY TLS AND MTLS COMPONENTS

Component	Purpose	Key Points	Common Issues
Certificate Chain	Proves server identity	Leaf cert + intermediate(s) + root	Missing intermediate, untrusted CA
SNI (Server Name Indication)	Selects the correct certificate	Client sends hostname in ClientHello	Wrong cert served, handshake failure
ALPN (Application Protocol Negotiation)	Negotiates application protocol	e.g. HTTP/1.1, HTTP/2, gRPC	Protocol mismatch, fallback issues
TLS Negotiation	Agrees on version and cipher	TLS 1.2 / 1.3, cipher suites	Unsupported version or cipher
Mutual Authentication (mTLS)	Client and server verify each other	Both sides present certificates	Client cert missing, untrusted CA
Session Resumption	Speeds up reconnection	Spesion ID or session ticket	Resumption failures after rotation
Trust Store	List of trusted root CAs	OS, application or custom store	Trust-store divergence across services

3. CERTIFICATE CHAIN EXAMPLE



4. COMMON TLS / MTLS FAILURE SCENARIOS

Scenario	What Happens	Likely Cause	How to Fix
Handshake failure	TLS connection fails	Unsupported version or cipher	Enable compatible TLS version / cipher
Unknown CA	Certificate not trusted	Missing intermediate or untrusted CA	Fix certificate chain / update trust store
Hostname mismatch	Certificate does not match hostname	Wrong cert or missing SAN	Use correct certificate with proper SAN
mTLS failure	Client or server rejects cert	Missing or invalid client certificate	Provide valid client cert and correct CA
Failure after rotation	Some services fail	Old certs cached or trust-store divergence	Synchronize rotation and clear caches
ALPN/protocol issue	Application fails after handshake	Protocol mismatch (e.g. HTTP/2 vs 1.1)	Ensure both sides support same protocol

5. TLS VS MTLS COMPARISON

Feature	TLS (One-Way)	mTLS (Mutual)
Authentication	Server only	Server and client
Client Certificate	Not required	Required
Use Cases	Web traffic, public APIs	Service-to-service, internal APIs
Security Level	Encrypts and verifies server	Encrypts and verifies both sides
Complexity	Lower	Higher (certificate management)

6. USEFUL DIAGNOSTIC COMMANDS

```
1 openssl s_client -connect example.com:443 -servername example.com
  # Test TLS handshake (with SNI)
2 openssl s_client -connect example.com:443 -showcerts
  # Xequret
3 openssl x509 -in cert.pem -text -noout        # Inspect certificate
4 curl -v https://example.com                   # See TLS details
5 curl --cacert ca.pem https://example.com      # Test with custom CA
6 openssl verify -CAfile ca.pem cert.pem        # Verify certificate chain
7 openssl s_client -connect service:443         # Test mTLS
  -cert client.pem -key client.key
```

SENIOR ENGINEER INSIGHT

- Most TLS issues are configuration, trust or certificate chain related, not application bugs.
- Certificate rotations often fail because of missed intermediates, cached certs, or unsynchronized trust stores.
- Always validate from both sides (client and server).
- In microservices, inconsistent trust stores across containers and environments are a common cause of partial failures.

8. REAL-WORLD TIPS

- Use a proper certificate chain (include intermediates).
- Monitor certificate expiry and set alerts.
- Automate certificate rotation and test in staging.
- Clear caches after rotation (applications, proxies, service mesh).
- Standardize TLS versions and cipher suites across services.
- Document and manage trust stores consistently.

9. COMMON MISTAKES

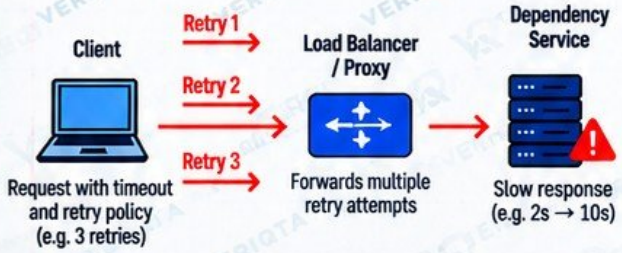
- Not sending intermediate certificates.
- Using self-signed certificates in production.
- Ignoring hostname/SAN mismatch.
- Inconsistent trust stores across services.
- Not testing mTLS from both sides.
- Assuming session resumption always works.
- Rotating certificates without coordinating all services.

Timeouts, Retries and Cascading Network Failure

Engineer deadline propagation, bounded retries, exponential backoff, jitter, retry budgets, circuit breakers, and connection reuse. Understand how client behavior can amplify outages.

Production Case: A brief dependency slowdown becomes a platform-wide incident.

1. HOW TIMEOUTS AND RETRIES AMPLIFY FAILURES



Cascading Network Failure

- Increased load
- Queue buildup
- Thread exhaustion
- More timeouts
- System-wide impact

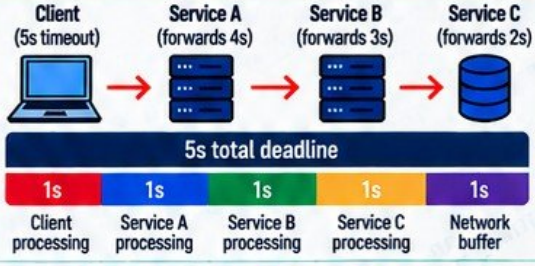
2. KEY CONCEPTS

Concept	What It Means	Why It Matters
Timeout	Maximum time to wait for a response	Prevents hanging requests
Retry	Repeat a failed request	Can increase load if not bounded
Exponential Backoff	Increase wait time between retries	Reduces pressure on struggling services
Jitter	Randomize retry delay	Prevents synchronized retry storms
Retry Budget	Limit total number of retries	Controls overall load during incidents
Circuit Breaker	Stop sending requests after failures	Prevents cascading failures
Connection Reuse	Keep connections alive (keep-alive, pooling)	Reduces latency and connection overhead
Deadline Propagation	Pass remaining time across service calls	Prevents downstream timeouts from exceeding upstream timeout

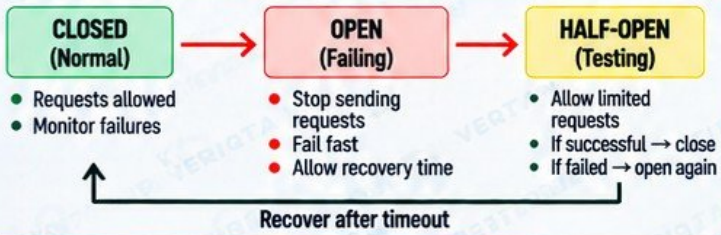
3. RETRY STRATEGIES (COMPARISON)

Strategy	How It Works	Advantages	Disadvantages
No Retry	Fail immediately	Simple, no extra load	Fails on transient issues
Fixed Retry	Retry after fixed delay (e.g. 1s)	Easy to implement	Can cause retry storms
Exponential Backoff	Increase delay (e.g. 1s, 2s, 4s, 8s)	Reduces load on failing service	Slower recovery
Exponential Backoff + Jitter	Randomize delay	Prevents synchronized retries	Slightly more complex
Retry Budget	Limit total retries (e.g. max 3 or 10%)	Controls retry load during incidents	Requires tuning

4. DEADLINE PROPAGATION (EXAMPLE)



5. CIRCUIT BREAKER STATES



6. COMMON FAILURE SCENARIO (CASCADE)

1. A downstream service becomes slow (e.g. 200ms → 5s).
2. Clients retry multiple times with short timeouts.
3. Load on the downstream service increases dramatically.
4. More requests time out, causing additional retries.
5. Upstream services become saturated (threads, connections, CPU).
6. Queues build up in proxies and load balancers.
7. Healthy services also start failing due to resource exhaustion.
8. A brief slowdown escalates into a platform-wide incident.

7. USEFUL DIAGNOSTIC COMMANDS

```
curl -v --max-time 5 https://example.com          # Test timeout behavior
curl -v --retry 3 --retry-delay 1 https://example.com # Test retries
time curl https://example.com                     # Measure response time
ss -t | grep ESTAB | wc -l                        # Check active connections
netstat -s | grep retrans                         # Check retransmissions
tcpdump -nn -i eth0 host <ip>                     # Capture retries/timeouts
openssl s_client -connect example.com:443         # Test TLS handshake
kubectl logs <service> | grep timeout             # Check timeout logs
```

8. BEST PRACTICES

- Use reasonable timeouts based on real latency.
- Implement bounded retries with exponential backoff and jitter.
- Set and enforce retry budgets.
- Use circuit breakers for failing dependencies.
- Propagate deadlines across service boundaries.
- Monitor and alert on timeout rates, retries and latency.
- Reuse connections with keep-alive and connection pooling.
- Test failure scenarios regularly (chaos engineering).

9. COMMON MISTAKES

- Setting timeouts too high or too low.
- Unlimited or aggressive retries.
- No jitter on retries (causes retry storms).
- Not using circuit breakers.
- Ignoring deadline propagation.
- Oversubscribing connections (no pooling).
- Not monitoring retry rates.
- Assuming client retries are harmless.
- Not testing slow dependency scenarios.

SENIOR ENGINEER INSIGHT

Retries are not free. They multiply load and can turn a small issue into a major outage.

- Always design client behaviour with the entire system in mind, not just the success of a single request.
- Observe, measure, and set budgets for retries and timeouts.

REAL-WORLD TIP

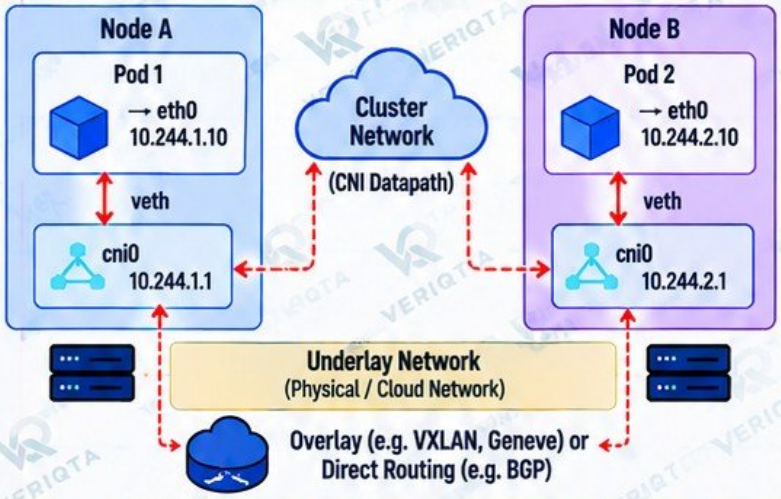
- Start with latency measurements from production.
- Set timeouts slightly above the P99 latency of the dependency.
- Use exponential backoff with jitter (e.g. 100ms, 200ms, 400ms).
- Implement and test circuit breakers.
- Monitor retries, timeout errors and connection pool saturation.
- Run game days to validate your configuration.

Pod Networking: Namespaces, veth, CNI and Packet Flow

Trace Pod-to-Pod traffic through network namespaces, virtual Ethernet pairs, node routing, CNI datapaths, overlays, and direct routing. Identify where packet loss occurs.

Production Case: Pods communicate on the same node but fail across nodes.

1. POD NETWORKING ARCHITECTURE (OVERVIEW)



2. PACKET FLOW (POD-TO-POD ACROSS NODES)

1	Packet leaves Pod 1	Sent from **eth0** in Pod 1 (10.244.1.10) to Pod 2 (10.244.2.10).
2	veth pair	Traffic goes from Pod network namespace to veth pair on the node.
3	CNI bridge / datapath	Packet enters **cni0** (or CNI-specific interface like **cali***, **flannel.1**, **weave**, etc).
4	Routing decision	Node routes traffic to the destination Pod network (remote node).
5	CNI encapsulation (if overlay)	Packet is encapsulated (e.g. VXLAN) and sent over the underlay network.
6	Underlay network	Packet traverses the physical or cloud network to the destination node.
7	Decapsulation	Remote node decapsulates the packet (if overlay).
8	Delivery to Pod 2	Packet is forwarded to **cni0**, then veth, and finally to **eth0** in Pod 2.

3. KEY COMPONENTS AND THEIR ROLE

Component	Purpose	Key Concepts	Common Issues
Network Namespace	Isolates Pod network stack	Each Pod has its own network namespace	Misconfigured routes or DNS
veth Pair	Connects Pod to node network	One end in Pod, one end on host	veth down or MTU mismatch
CNI Plugin	Sets up networking for Pods	Examples: Calico, Flannel, Cilium, Weave	CNI misconfiguration or agent failure
Pod Network (Overlay/Direct)	Connects Pods across nodes	Overlay (VXLAN, Geneve) or direct routing (BGP)	Encapsulation, routing or firewall issues
Node Routing	Routes traffic between Pod networks	Uses kernel routing table and CNI rules	Missing routes or asymmetric routing
Underlay Network	Physical/cloud infrastructure	VPC, VNet, on-prem network	Network ACL, security group or MTU issues

4. COMMON FAILURE POINTS

- CNI pods not running or crashing.
- Incorrect routing between nodes.
- Firewall or security group blocking overlay traffic (e.g. UDP 4789 for VXLAN).
- MTU mismatch causing packet drops (especially with overlays).
- Asymmetric routing.
- Node-to-node connectivity issues.
- IP address conflicts or IPAM exhaustion.
- CNI misconfiguration (BGP, VXLAN, routes).
- Network policies blocking traffic.
- DNS working but pod IP routing failing.

5. SAME NODE vs ACROSS NODES

Scenario	Traffic Path	Common Result	Typical Cause
Pods on the same node	Pod → veth → cni0 → veth → Pod	Works (usually)	Local CNI setup is correct
Pods on different nodes	Pod → veth → cni0 → overlay/underlay → remote cni0 → Pod	Fails	CNI overlay, routing, firewall, MTU or node-to-node issue

6. USEFUL DIAGNOSTIC COMMANDS

```
kubectl get pods -n kube-system          # Check CNI pods
kubectl logs -n kube-system <cni-pod>    # View CNI logs
ip netns list                            # List network namespaces
ip netns exec <ns> ip a                  # Check Pod network namespace
ip link show                             # View veth interfaces
ip route show                            # Check node routing table
ping <pod-ip>                            # Test connectivity
traceroute <pod-ip>                      # Trace packet path
tcpdump -i any host <pod-ip>             # Capture packets
iptables -L -n -v                        # Check iptables rules
```

7. BEST PRACTICES

- Validate CNI health and logs.
- Ensure correct node-to-node routing and firewall rules.
- Use consistent MTU across the cluster.
- Monitor CNI metrics and pod networking latency.
- Use network policies intentionally.
- Implement proper IP address management (IPAM).
- Test both same-node and cross-node pod connectivity.
- Document CNI configuration and changes.

8. REAL-WORLD TROUBLESHOOTING STEPS

1. Verify Pods and CNI pods are running.
2. Check Pod IPs and node IPs.
3. Test connectivity (ping, traceroute).
4. Capture packets on both nodes.
5. Check MTU and look for fragmentation.
6. Verify routing tables and CNI configuration.
7. Check firewall/security group rules.
8. Review network policies.
9. Identify where packets are dropped.
10. Fix the underlying network or CNI issue and retest.

SENIOR ENGINEER INSIGHT

- If Pods can communicate on the same node but not across nodes, the issue is almost always in the overlay/underlay network, routing, MTU or firewall rules.
- Always test with ping, traceroute and packet capture to find the exact drop point.
- CNI issues can be subtle and often require checking both node configuration and CNI pod logs.

REAL-WORLD TIP

- For VXLAN (Flannel), ensure UDP 4789 is open.
- For Calico (BGP), verify BGP peering and routes.
- Keep MTU consistent (e.g. 1450 for cloud overlays).
- Monitor CNI and node networking metrics.
- Use network policies carefully to avoid accidental blocks.

COMMON MISTAKES

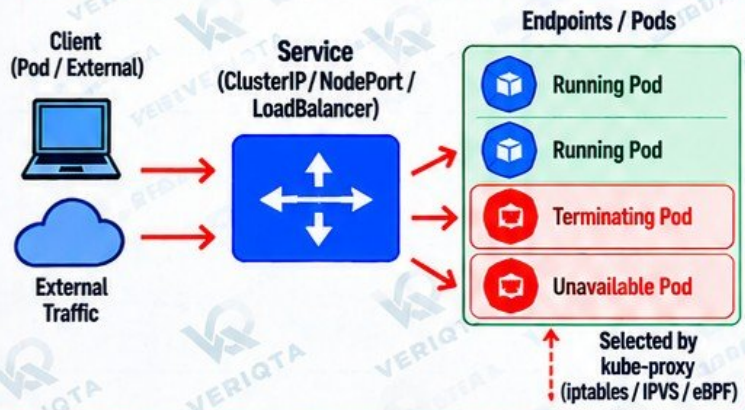
- Only testing from one node.
- Ignoring MTU issues.
- Forgetting firewall/security group rules.
- Not checking CNI pod logs.
- Assuming DNS issues are always the cause.
- Not validating routing tables on both nodes.

Kubernetes Services: iptables, IPVS and eBPF

Compare service forwarding implementations, endpoint selection, connection tracking, kube-proxy behavior, eBPF-based routing, and EndpointSlice propagation.

Production Case: A Service intermittently routes requests to terminating or unavailable Pods.

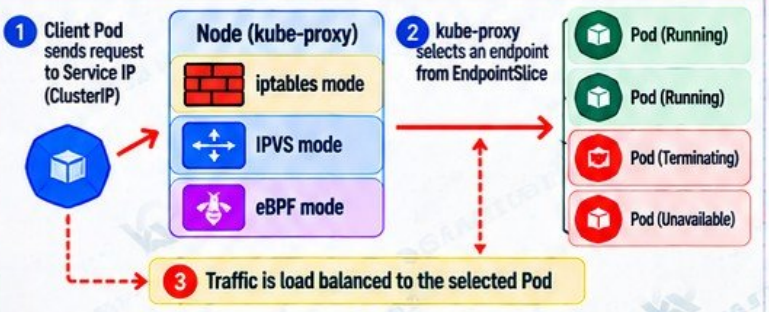
1. KUBERNETES SERVICE ARCHITECTURE (OVERVIEW)



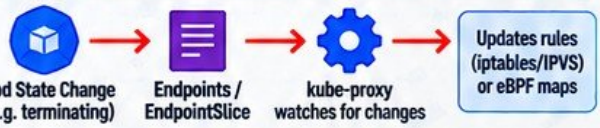
2. SERVICE FORWARDING IMPLEMENTATIONS

iptables (legacy)	IPVS (default in many clusters)	eBPF (Cilium, kube-proxy eBPF mode, etc.)
• Uses iptables rules in the node kernel. • Service traffic DNAT to Pod IP. • Simple and widely compatible. • Can be inefficient at large scale. • Endpoint updates modify iptables rules.	• Uses Linux IPVS for load balancing. • More efficient and scalable than iptables. • Supports multiple scheduling algorithms (rr, lc, sh, etc.). • Works with connection tracking (conntrack). • Updates handled by kube-proxy.	• Uses eBPF programs in the kernel. • Direct routing without iptables or IPVS. • Faster, more scalable, and more observable. • Supports advanced features (L7, security, policy). • Endpoint updates handled through eBPF maps.

3. PACKET FLOW THROUGH A KUBERNETES SERVICE



4. ENDPOINTS, ENDPOINTSLICE AND PROPAGATION



- **EndpointSlice** is used for scalable endpoint management.
- kube-proxy watches EndpointSlice for changes.
- There can be a delay between Pod state change and rule/map update.
- Terminating Pods may still receive traffic during this window (if not configured correctly).
- Readiness probes, terminationGracePeriodSeconds and proper lifecycle handling reduce the risk.

5. ENDPOINT SELECTION AND LOAD BALANCING

Aspect	iptables	IPVS	eBPF
Selection Method	Random (statistic)	Multiple algorithms (rr, lc, sh, etc.)	Custom, programmable (eBPF)
Connection Tracking	Uses conntrack	Uses conntrack with IPVS	Can bypass or use conntrack (configurable)
Session Affinity	Supported	Supported	Supported (advanced)
Health Checking	Relies on readiness probes (no active check)	Relies on readiness probes (no active check)	Can integrate L7 checks (with Cilium, etc.)
Scalability	Lower (large rule sets)	Higher	Highest (no rule churn)

6. COMMON FAILURE CAUSES

Cause	Explanation
Stale endpoints	kube-proxy has not yet removed terminating Pods.
Readiness probe misconfiguration	Pod marked ready before it can actually serve traffic.
Slow EndpointSlice propagation	Delay in updating endpoints across nodes.
Connection reuse	Existing connections continue to terminating Pods until closed.
Incorrect kube-proxy mode	iptables mode at large scale may cause delays.
Node-level issues	Network, conntrack or resource exhaustion on a node.
CNI / eBPF policy	eBPF policies blocking traffic to certain Pods.
Application behavior	Long-lived connections to Pods that are terminating.

7. USEFUL DIAGNOSTIC COMMANDS

```
1  kubectl get svc -A                              # List all Services
2  kubectl get endpoints <svc>                     # View endpoints
3  kubectl get endpointslices                      # View EndpointSlices
4  kubectl describe endpointslice <name>           # Check endpoint details
5  iptables -t nat -L -n | grep <service-ip>       # Check iptables rules
6  ipvsadm -Ln                                     # View IPVS rules (if used)
7  ss -tnp | grep <service-ip>                     # Check active connections
8  conntrack -L | grep <service-ip>                # Check connection tracking
9  bpftool map show                                # View eBPF maps (Cilium, etc.)
10 bpftool prog show                               # View eBPF programs
```

8. BEST PRACTICES

- ✓ Use readiness and liveness probes correctly.
- ✓ Set appropriate terminationGracePeriodSeconds.
- ✓ Use preStop hooks for graceful shutdown.
- ✓ Monitor EndpointSlice propagation delays.
- ✓ Tune conntrack settings for high load.
- ✓ Use IPVS or eBPF for large clusters.
- ✓ Implement connection draining and proper client retry logic.
- ✓ Monitor kube-proxy, CNI and node networking.
- ✓ Use observability (metrics, logs, tracing) to detect stale endpoints.

9. COMMON MISTAKES

- ✗ Sending traffic to terminating Pods.
- ✗ Relying only on readiness probes without proper shutdown handling.
- ✗ Ignoring EndpointSlice delays.
- ✗ Not considering connection reuse.
- ✗ Using iptables mode at large scale without tuning.
- ✗ Not monitoring conntrack or node resource usage.
- ✗ Assuming kube-proxy updates are instant.
- ✗ Ignoring CNI or eBPF network policies.
- ✗ Not testing failover and rolling updates in production-like conditions.

SENIOR ENGINEER INSIGHT

- A Service is only as healthy as its endpoint management.
- Always consider the delay between Pod state change and kube-proxy rule/map update.
- Use observability to verify where traffic is actually going.
- Terminating Pods can still receive traffic — design for graceful shutdown.

REAL-WORLD TIP

- During incidents, check endpoints and conntrack first.
- Use IPVS or eBPF for high-scale environments.
- Validate readiness probes and termination settings.
- Test rolling updates and node drain scenarios regularly.

PRODUCTION CHECKLIST

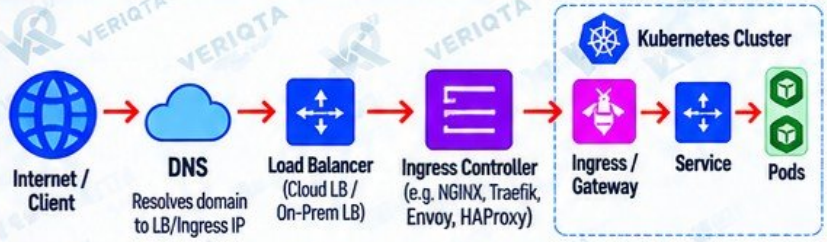
- Verify Service endpoints are correct.
- Confirm terminating Pods are not receiving traffic.
- Check kube-proxy mode and configuration.
- Monitor conntrack, node resources and CNI.
- Test failover and rolling updates.

Ingress and Gateway API: Production Traffic Engineering

Examine ingress controllers, Gateway API resources, traffic ownership, TLS boundaries, route precedence, weighted routing, readiness, and zero-downtime cutovers.

Production Case: A rollout causes intermittent 503 errors despite healthy Pods.

1. FROM INTERNET TO POD (HIGH-LEVEL FLOW)



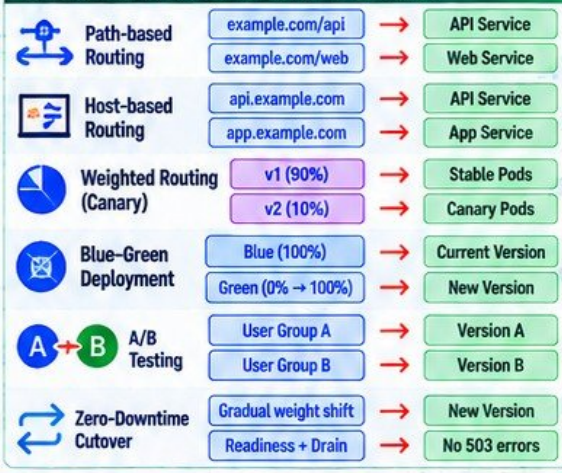
2. INGRESS vs GATEWAY API (KEY DIFFERENCES)

Aspect	Ingress (v1)	Gateway API (v1)
Scope	Kubernetes-specific	Kubernetes-native, more extensible
Resources	Ingress	Gateway, GatewayClass, HTTPRoute, TLSRoute, etc.
Design	Simpler, widely adopted	More expressive and flexible
Traffic Ownership	Implicit (controller-specific)	Explicit via Gateway and Routes
Features	Basic routing, TLS, annotations	Advanced routing, traffic policies, multi-tenant
Status	Stable (v1)	Next-generation, recommended for new setups

3. KEY COMPONENTS AND THEIR ROLE

Component	Purpose	Key Configuration	Common Issues
Ingress Controller	Implements ingress rules and routes traffic	NGINX, Traefik, Envoy, HAProxy, cloud controller	Misconfiguration, resource limits, reload delays
Gateway	Defines entry point for traffic (Gateway API)	GatewayClass, listeners, TLS config	Incorrect listeners, wrong TLS mode
HTTPRoute / TLSRoute	Defines routing rules (Gateway API)	Hostnames, paths, backends, weights	Route precedence, conflicting rules
Service	Stable endpoint for Pods	ClusterIP, port mapping	Points to wrong Pods, no ready endpoints
TLS Termination	Handles HTTPS decryption (edge or passthrough)	TLS certificates, SNI, secret management	Wrong certificate, SNI mismatch
Backend Pods	Handles application traffic	Readiness probes, resource limits	Not ready, slow startup, terminating pods receiving traffic

4. TRAFFIC ENGINEERING PATTERNS



5. TLS BOUNDARIES AND TERMINATION MODES



Edge Termination
- TLS terminates at LB or ingress controller
- Traffic to backend over HTTP

Re-encryption
- TLS terminates at ingress and re-encrypts to backend
- End-to-end encryption within cluster

Passthrough
- TLS passes through ingress to backend
- Used for mTLS or custom TLS handling

6. COMMON CAUSES OF 503 ERRORS (EVEN WITH HEALTHY PODS)

Cause	Explanation
Not Ready Endpoints	Ingress routes to Pods before they are ready
Propagation Delay	Endpoint/EndpointSlice update not yet reflected
Configuration Reload	Ingress controller reloading config, temporary drop
Incorrect Readiness Probe	Pods marked unready or flapping
Draining / Terminating Pods	Traffic still sent to terminating Pods
Route Precedence	Conflicting or overlapping routes
LB Health Check Mismatch	LB marks node/ingress unhealthy
Session Affinity Issues	Sticky sessions sent to unavailable Pods
Rate Limiting / WAF	Requests blocked or throttled by ingress/LB
Resource Limits	Ingress controller CPU/memory saturation

7. USEFUL DIAGNOSTIC COMMANDS

```
kubectl get ingress -A                          # List all ingresses
kubectl get gateway,httproute -A                # List Gateway API resources
kubectl describe ingress <name>                 # Check ingress details
kubectl get endpoints,endpointslice             # Check service endpoints
kubectl logs -n <ns> <ingress-pod>              # View ingress controller logs
curl -vk https://example.com                    # Test TLS and routing
openssl s_client -connect example.com:443 -servername example.com # Check certificate
kubectl get pods -n <ns>                        # Check pod readiness
kubectl get svc -A                              # Check services
kubectl get events -A --sort-by=.lastTimestamp  # Check recent events
```

8. BEST PRACTICES

- Use readiness and liveness probes correctly.
- Validate ingress/Gateway configuration before rollout.
- Use gradual rollouts with weighted routing.
- Ensure proper TLS certificate management.
- Monitor ingress controller metrics and logs.
- Use EndpointSlice and check propagation delays.
- Drain connections before terminating Pods.
- Test routing rules and precedence.
- Use observability (metrics, tracing, logs).
- Implement zero-downtime deployment strategies.

9. COMMON MISTAKES

- Relying only on readiness probes.
- Ignoring EndpointSlice propagation.
- Misconfigured TLS certificates or SNI.
- Overlapping or conflicting routes.
- Not draining terminating Pods.
- Assuming instant config reloads.
- Using incorrect ingress annotations.
- Not testing weighted routing.
- Ignoring LB health check configuration.
- Not monitoring ingress controller resources.

SENIOR ENGINEER INSIGHT

503 errors often indicate a traffic engineering or configuration issue, not always a problem with the application.

- Always verify the entire path: DNS → LB → ingress/gateway → service → endpoints → pods.
- Use gradual rollouts, proper readiness probes, and observability to prevent and quickly identify traffic issues.

REAL-WORLD TIP

During rollouts, monitor EndpointSlice updates and ingress controller logs.

- Use weighted routing and small percentage rollouts.
- Always test TLS configuration and certificate expiry.
- Keep ingress controller resources adequately sized.

PRODUCTION CHECKLIST

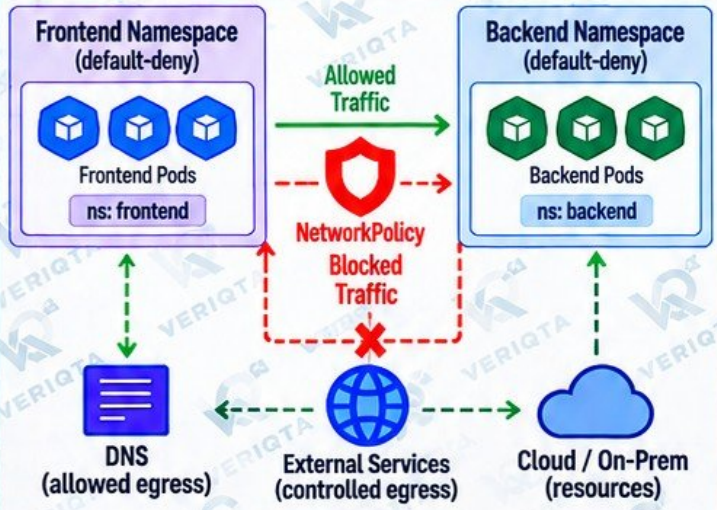
- Validate routes, TLS, and certificates.
- Confirm endpoints are ready before rollout.
- Monitor ingress metrics, logs, and events.
- Test failover, rollback, and zero-downtime cutover.
- Verify no traffic is sent to terminating Pods.

NetworkPolicy and Zero-Trust Cluster Segmentation

Engineer default-deny controls, ingress and egress policy, DNS allowances, workload identity boundaries, and CNI enforcement limitations. Validate effective isolation.

Production Case: A security policy blocks an essential dependency during deployment.

1. ZERO-TRUST CLUSTER SEGMENTATION (OVERVIEW)



2. NETWORKPOLICY COMPONENTS

Component	Purpose	Key Configuration	Common Issues
Default-Deny Policy	Starts with no traffic allowed	Empty ingress and egress rules	Forgets to allow essential traffic (e.g. DNS)
Ingress Rules	Controls incoming traffic to Pods	from (podSelector, namespaceSelector, IPBlock), ports	Missing required sources or ports
Egress Rules	Controls outgoing traffic from Pods	to (podSelector, namespaceSelector, IPBlock), ports	Blocks DNS, APIs or external services
DNS Allowances	Allows name resolution	Egress to kube-dns or CoreDNS (UDP/TCP 53)	DNS blocked, Pods cannot resolve names
Workload Identity Boundaries	Combine with service accounts and identity	Namespace and label based segmentation	Overly broad access between workloads
CNI Enforcement Limitations	Implementation dependent	Calico, Cilium, Weave, others (L3/L4/L7)	Some CNIs do not enforce all policy types or host traffic

3. EXAMPLE NETWORKPOLICY CONFIGURATIONS

Default-Deny (both ingress and egress)

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: app
spec:
  podSelector: {}
  policyTypes:
  - Ingress
  - Egress
  ingress: []
  egress: []
```

Allow DNS Egress (CoreDNS)

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dns
  namespace: app
spec:
  podSelector: {}
  policyTypes:
  - Egress
  egress:
  - to:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: kube-system
      podSelector:
        matchLabels:
          k8s-app: kube-dns
    ports:
    - protocol: UDP
      port: 53
    - protocol: TCP
      port: 53
```

4. TRAFFIC SCENARIOS (ALLOWED vs BLOCKED)

Scenario	Allowed?	Reason
Frontend → Backend (on allowed port)	✓	Matches ingress rule (pod/namespace selector and port)
Frontend → Backend (on other port)	✗	Port not allowed in ingress rule
Pod → DNS (CoreDNS)	✓	Allowed by DNS egress policy
Pod → External API (on 443)	✓	Allowed by egress rule to IPBlock or domain via egress gateway
Pod → External API (not allowed)	✗	No matching egress rule
Cross-namespace (unintended)	✗	Blocked by default-deny
Same namespace (if allowed)	✓	Allowed if policy permits
Host network traffic	✗	Not controlled by standard NetworkPolicy (CNI dependent)

5. VALIDATING EFFECTIVE ISOLATION

```
1 kubectl get networkpolicy -A                          # List all policies
2 kubectl describe networkpolicy <name>                 # View policy details
3 kubectl get pods -n <ns> --show-labels                # Check labels
4 kubectl exec -it <pod> -- nslookup <service>          # Test DNS
5 kubectl exec -it <pod> -- curl -v <service>           # Test allowed traffic
6 kubectl exec -it <pod> -- curl -v https://<external>  # Test egress
7 kubectl get endpoints -n kube-system                  # Check CoreDNS endpoints
8 kubectl logs -n kube-system -l k8s-app=kube-dns       # Check DNS logs
```

6. COMMON FAILURE CAUSE: BLOCKED DEPENDENCY



Result: Deployment fails, application cannot reach dependency.

7. BEST PRACTICES

- Start with default-deny and explicitly allow required traffic.
- Always allow DNS for name resolution.
- Use namespace and label based segmentation.
- Document and version your NetworkPolicies.
- Test connectivity during deployment.
- Monitor and alert on denied traffic.
- Understand your CNI's enforcement capabilities and limitations.
- Use workload identity (service accounts) with least privilege.

8. SENIOR ENGINEER INSIGHT

- NetworkPolicy enforces L3/L4 rules; it is not a full zero-trust solution on its own.
- Always include DNS, monitoring, logging and necessary external dependencies in your allow rules.
- Validate policies in a staging environment before production.
- Use observability (flow logs, metrics) to detect unintended blocks.
- Be aware of CNI limitations (e.g. host network, certain protocols, L7 enforcement).

9. COMMON MISTAKES

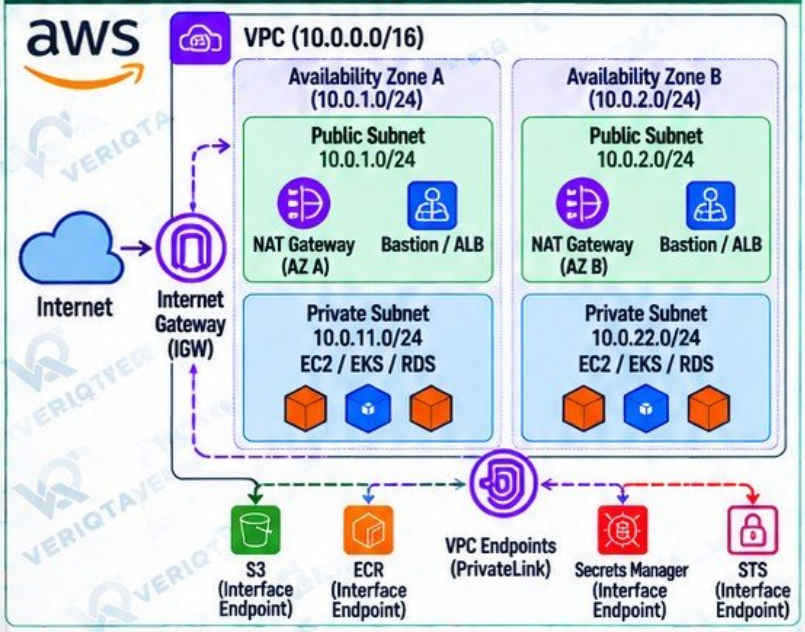
- ✗ Forgetting to allow DNS.
- ✗ Overly broad allow rules (defeats isolation).
- ✗ Blocking essential external services.
- ✗ Not testing policies before deployment.
- ✗ Assuming NetworkPolicy controls host network traffic.
- ✗ Ignoring CNI limitations.
- ✗ Not monitoring denied traffic.
- ✗ Using only IP-based rules without labels.

AWS VPC Architecture: Routes, Endpoints and Egress

Analyze subnet boundaries, route tables, security groups, NACLs, NAT gateways, private endpoints, IP address exhaustion, and cross-AZ costs.

Production Case: Private workloads lose access to critical cloud services during an AZ failure.

1. AWS VPC ARCHITECTURE OVERVIEW



2. KEY COMPONENTS

Component	Purpose	Key Points
VPC	Isolated network in AWS	Defined by CIDR, spans multiple AZs
Subnet	Network segment within an AZ	Public or private, associated with route table
Route Table	Controls traffic routing	Defines destination and next hop (IGW, NAT, VGW, etc.)
Internet Gateway (IGW)	Provides internet access to public subnets	Attached to VPC
NAT Gateway	Allows private subnets to access the internet	AZ-specific, managed, high availability requires one per AZ
Security Group (SG)	Stateful firewall at instance level	Allow rules only, no deny, stateful
Network ACL (NACL)	Stateless firewall at subnet level	Allow and deny rules, affects inbound and outbound traffic
VPC Endpoints (PrivateLink)	Private access to AWS services	Interface endpoints (ENI) or gateway endpoints (S3, DYNAMODB)
Elastic IP	Static public IP	Useful for NAT, bastion, etc.

3. ROUTE TABLE EXAMPLES

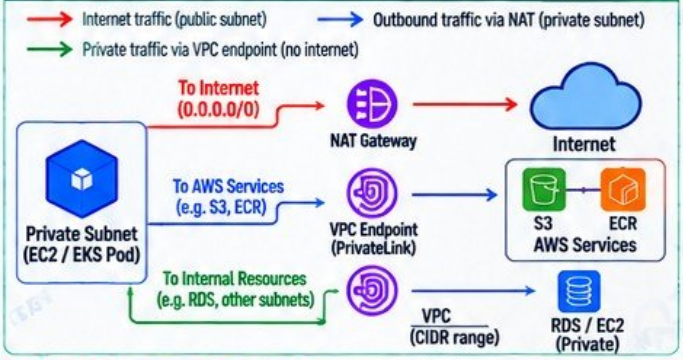
Public Subnet Route Table (rtb-public)

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	igw-xxxxxxxx

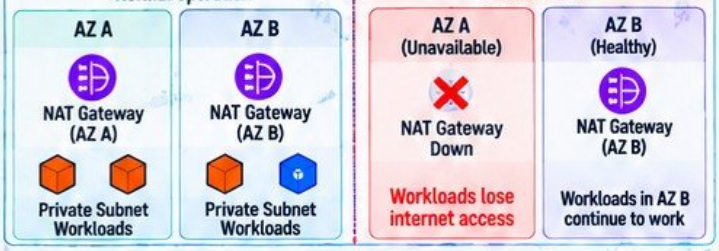
Private Subnet Route Table (rtb-private)

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	nat-xxxxxxxx
com.amazonaws.region.s3	vpce-xxxxxxxx
com.amazonaws.region.ecr	vpce-xxxxxxxx

4. TRAFFIC FLOWS



5. HIGH AVAILABILITY AND AZ FAILURE SCENARIO



6. COMMON FAILURE CAUSES

Cause	Explanation
Single NAT Gateway	If the AZ fails, private workloads in that AZ lose internet access.
Missing VPC Endpoints	Cannot reach AWS services when NAT or internet is unavailable.
Incorrect Route Table	Missing or wrong routes to NAT, IGW or endpoints.
Security Group Blocking	Outbound traffic blocked to required ports/services.
NACL Misconfiguration	Stateless rules blocking return traffic.
IP Address Exhaustion	Not enough IPs in subnet or ENI limits reached.
Cross-AZ Traffic Costs	Traffic between AZs incurs additional charges.
Endpoint Policy Restriction	VPC endpoint policy denies required AWS services.

7. USEFUL DIAGNOSTIC COMMANDS (FROM EC2 / POD)

```
ip route                              # Check routing table
curl -I https://aws.amazon.com        # Test internet access
nslookup s3.amazonaws.com             # Test DNS resolution
curl -I https://s3.amazonaws.com      # Test S3 access
traceroute 1.1.1.1                    # Trace network path
aws s3 ls                             # Test AWS service access (with IAM)
iptables -L -n                        # Check local firewall rules
nc -vz s3.amazonaws.com 443           # Test port connectivity
```

8. BEST PRACTICES

- Use a NAT Gateway in each AZ for high availability.
- Use VPC endpoints for critical AWS services.
- Apply least-privilege security group and NACL rules.
- Plan IP address space to avoid exhaustion.
- Test AZ failure scenarios regularly.
- Monitor NAT Gateway and endpoint health.
- Use centralized egress design for compliance.
- Consider cross-AZ costs in architecture design.

9. COMMON MISTAKES

- Relying on a single NAT Gateway.
- Forgetting DNS resolution (port 53).
- Blocking required AWS endpoints.
- Using overly restrictive NACLs.
- Not testing AZ failure scenarios.
- Ignoring IP address limits (ENI, subnet).
- Unexpected cross-AZ data transfer costs.
- Using instance-level public IPs instead of proper egress design.

SENIOR ENGINEER INSIGHT

- Design for AZ failure, not just instance failure.
- Use VPC endpoints to reduce dependency on internet/NAT.
- Always validate connectivity from private subnets during incidents.
- Monitor IP usage and set alerts before exhaustion.

REAL-WORLD TIP

- Use AWS Reachability Analyzer to test network paths.
- Implement VPC Flow Logs for traffic visibility.
- Tag and document all route tables, subnets and endpoints.
- Regularly review NAT Gateway and endpoint costs.

PRODUCTION CHECKLIST

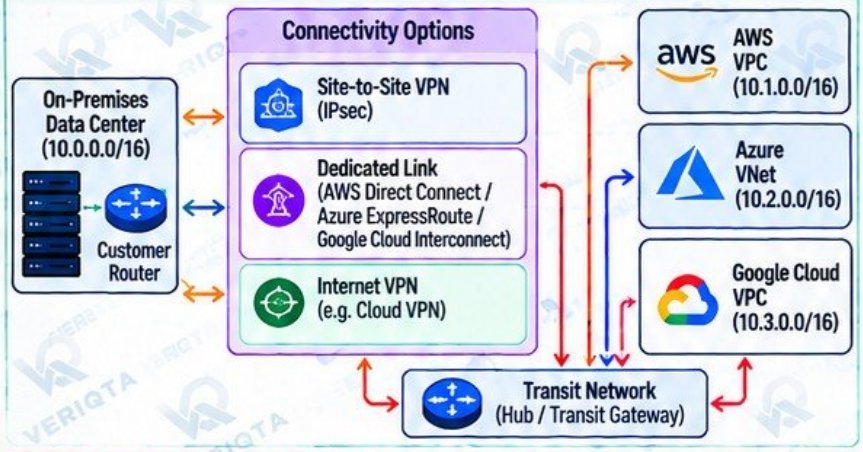
- NAT Gateways in each AZ and monitored.
- Required VPC endpoints configured and tested.
- Route tables correctly associated.
- Security groups and NACLs validated.
- AZ failure test completed and documented.
- IP address utilization monitored.

Hybrid and Multi-Cloud: BGP, VPN and Route Control

Examine BGP route advertisements, overlapping CIDRs, VPN and dedicated connectivity, route propagation, failover convergence, and centralized transit architectures.

Production Case: A route change silently diverts production traffic into a black hole.

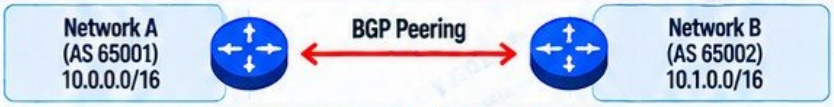
1. HYBRID AND MULTI-CLOUD ARCHITECTURE (OVERVIEW)



2. KEY CONCEPTS AND COMPONENTS

Component	Purpose	Key Points
BGP	Dynamic routing between networks	Advertises and learns routes, path selection, AS path, communities, route filters.
VPN (IPsec)	Encrypted connectivity over the internet	Used for hybrid and cross-cloud connectivity. Redundant tunnels recommended.
Dedicated Connectivity	Private, high performance links	AWS Direct Connect, Azure ExpressRoute, Google Cloud Interconnect.
Transit Architecture	Centralized routing across multiple networks/clouds	AWS Transit Gateway, Azure Virtual WAN, Google Cloud Network Connectivity Center.
Overlapping CIDRs	Conflicting IP ranges across networks	Requires NAT, route translation or IP readdressing.
Route Propagation	Distributes routes across devices	Control which routes are advertised and accepted.
Failover Convergence	Time to detect and switch to alternate path	BGP timers, route health checks, and proper design reduce downtime.

3. BGP ROUTE ADVERTISEMENT AND PATH SELECTION



BGP Advertises:

- Network prefixes (e.g. 10.0.0.0/16)
- AS path
- Next hop
- Attributes (local preference, MED, communities, etc.)

Path Selection (Simplified):

1. Highest local preference
2. Shortest AS path
3. Lowest MED
4. eBGP preferred over iBGP
5. Route filters and policies

4. ROUTE CONTROL AND POLICY EXAMPLES

```
# Example: Advertise only specific prefixes (Cisco-like)
ip prefix-list ONPREM seq 5 permit 10.0.0.0/16
route-map TO-CLOUD permit 10
  match ip address prefix-list ONPREM
  set local-preference 200
  neighbor 192.168.1.1 route-map TO-CLOUD out
# Example: Prevent accepting unwanted routes
ip prefix-list BLOCK seq 5 deny 0.0.0.0/0
route-map FROM-CLOUD permit 10
  match ip address prefix-list BLOCK
  neighbor 192.168.1.1 route-map FROM-CLOUD in
```

5. COMMON HYBRID AND MULTI-CLOUD TOPOLOGIES

1 Point-to-Point VPN



On-Prem ↔ Cloud (Site-to-Site VPN)

- Simple and low cost
- Suitable for small setups

2 Dedicated Connectivity



On-Prem ↔ Cloud (Direct Connect / ExpressRoute)

- High performance
- More reliable, lower latency

3 Transit Hub (Single Cloud)



Multiple VPCs/VNets via Transit Gateway

- Centralized route control
- Simpler management

4 Multi-Cloud Transit



Centralized hub across AWS, Azure and GCP

- Consistent routing policies
- Simplifies hybrid and multi-cloud

6. COMMON FAILURE CAUSES

Cause	Explanation
Incorrect route advertisement	Announcing the wrong prefix or next hop.
Overlapping CIDRs	Conflicting IP ranges cause traffic to go to the wrong network.
Missing or incorrect route filters	Accepting unintended routes (e.g. default route 0.0.0.0/0).
Route change or reconfiguration	New route silently diverts traffic.
BGP session loss	Failover not working or slow convergence.
Asymmetric routing	Return traffic takes a different path and is dropped.
VPN tunnel failure	No redundant tunnels or failover.

7. USEFUL DIAGNOSTIC COMMANDS

```
show ip bgp summary              # Check BGP peer status
show ip bgp                      # View learned and advertised routes
show ip route                    # Check routing table
traceroute <destination>         # Trace network path
ping <destination>               # Test reachability
curl -v <service-ip>             # Test application connectivity
ip a / ifconfig                  # Check network interfaces
tcpdump -i any port 179          # Capture BGP traffic
nslookup <domain>                # Test DNS resolution
mtr <destination>                # Continuous network diagnostic
```

8. BEST PRACTICES

- Use clear IP addressing plan and avoid overlapping CIDRs.
- Apply route filters and prefix lists.
- Use redundant VPN tunnels or dedicated links.
- Design centralized transit architecture.
- Monitor BGP sessions and route changes.
- Set appropriate BGP timers for faster convergence.
- Implement proper route summarization.
- Test failover scenarios regularly.
- Use monitoring and alerting for route changes.
- Document routing policies and network design.

9. COMMON MISTAKES

- Advertising unintended prefixes.
- Ignoring overlapping CIDRs.
- Not using route filters.
- Single point of failure for connectivity.
- Not monitoring BGP sessions.
- Assuming failover will be automatic.
- Using the same CIDR across multiple clouds.
- Not testing route changes in a safe environment.
- Missing or incorrect security controls on VPN connections.
- Ignoring cross-cloud data transfer costs.

SENIOR ENGINEER INSIGHT

- A small route change can have a massive impact.
- Always use route filters and monitor BGP updates.
- Test failover scenarios before production.
- Centralized transit architectures simplify control and reduce risk.

REAL-WORLD TIP

Use BGP communities and route tagging to control traffic flow and failover behavior.
Keep an out-of-band management path in case routing issues cut access.
Monitor route tables, BGP logs and flow logs for early detection.

PRODUCTION CHECKLIST

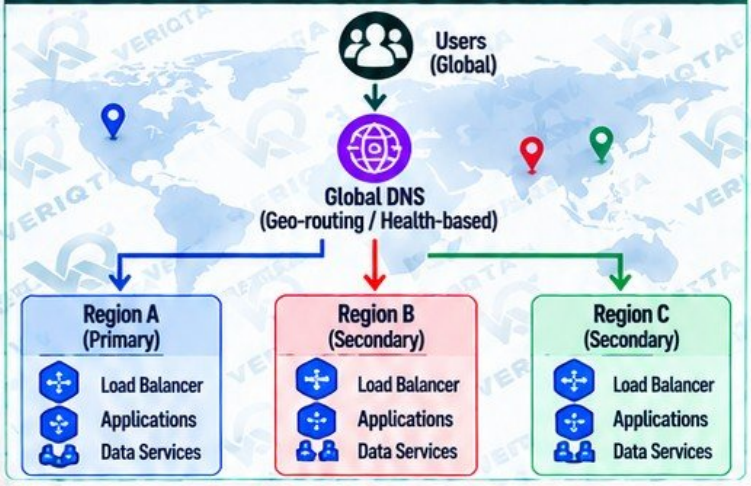
- Verify advertised and accepted routes.
- Confirm VPN / dedicated links are healthy.
- Check for overlapping CIDRs.
- Validate route filters and policies.
- Test failover and convergence.
- Monitor for unexpected route changes.

Multi-Region Traffic: Failover, Consistency and Recovery

Design global routing with DNS, anycast, regional load balancing, health signals, traffic steering, session affinity, and dependency-aware failover.

Production Case: Traffic fails over successfully, but the application remains unavailable because its data dependencies did not recover.

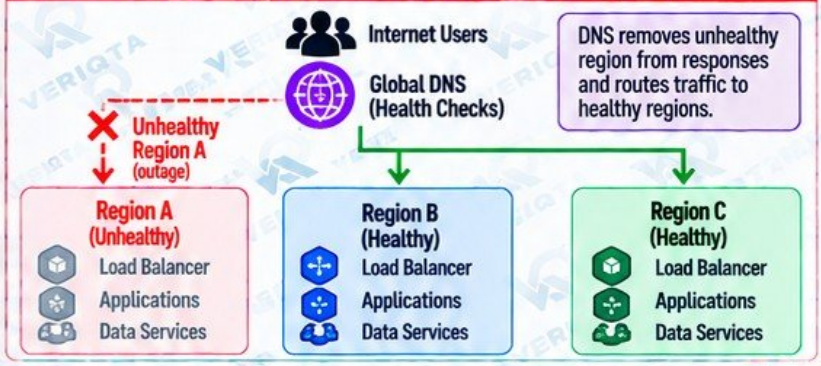
1. GLOBAL TRAFFIC ARCHITECTURE (OVERVIEW)



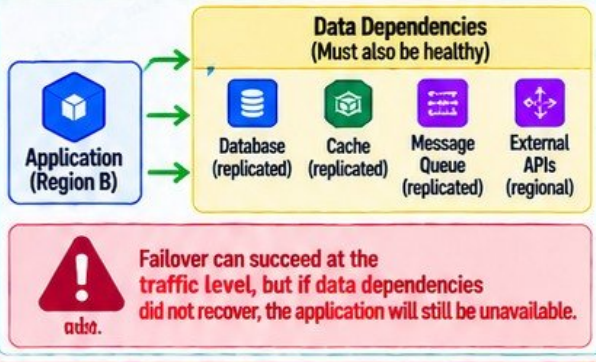
2. GLOBAL ROUTING METHODS

Method	How It Works	Use Cases	Key Considerations
DNS (Geo-routing)	Returns region-specific IP based on user location or health	Global traffic distribution	TTL, health checks, DNS propagation delay
Anycast	Same IP announced from multiple regions	Low-latency global entry point	Requires careful routing and health control
Global Load Balancer (e.g. AWS, Azure, GCP)	Routes traffic to healthy regional endpoints	Active-active or active-passive across-regions	Health signals, failover policies, cross-region costs
Application-level Routing	Built into application (e.g. service discovery)	Fine-grained control, session affinity	More complex to implement
Traffic Steering	Shifts traffic based on policies, latency, health or load	Maintenance, failure response	Requires real-time health data and automation

3. FAILOVER FLOW (DNS + REGIONAL LOAD BALANCING)



4. DATA DEPENDENCIES AND RECOVERY



5. CONSISTENCY MODELS IN MULTI-REGION

Model	Description	Advantages	Trade-offs
Strong Consistency	All regions see the same data immediately	Data accuracy	Higher latency, more complex
Eventual Consistency	Changes propagate over time	High availability, better performance	Temporary data inconsistency
Read-Local / Write-Global	Reads from local region, writes replicated globally	Lower latency	Conflict resolution needed
Active-Passive	One region active, others on standby	Simpler design	Failover time, underutilized resources
Active-Active	Multiple regions active	High availability, better performance	Complex data synchronization

6. COMMON FAILURE CAUSES

Cause	Explanation
Data dependency not recovered	Database, cache or message queue not available in the failover region.
Partial replication	Data not fully replicated or behind.
DNS propagation delay	Some users still routed to failed region (TTL not expired).
Misconfigured health checks	Unhealthy endpoints marked as healthy or vice versa.
Session affinity issues	User sessions tied to original region.
External dependency failure	Third-party services not available in failover region.
Network or routing misconfiguration	Incorrect routes, security groups or firewall rules.
Stateful workload challenges	State not replicated or fails to start in new region.

7. USEFUL DIAGNOSTIC COMMANDS

```
dig example.com                          # DNS resolution and IPs
dig +trace example.com                   # Trace DNS path
curl -I https://example.com              # Check endpoint health
nslookup example.com                     # Alterrative DNS lookup
mtr <region-ip>                          # Network path and latency
ping <region-ip>                         # Basic connectiity
traceroute <region-ip>                   # Trace route to region
aws route53 get-health-check-status      # AWS health check status
kubectl get endpoints                    # Application endpoints
curl -v <service-endpoint>               # Verbose request for debugging
```

8. BEST PRACTICES

- Design for dependency-aware failover.
- Use health checks for both application and data services.
- Test failover regularly across all dependencies.
- Use appropriate DNS TTL values.
- Implement automated traffic steering.
- Consider multi-region data replication.
- Use session management strategies.
- Monitor end-to-end availability (application and dependencies).
- Document and test recovery procedures.

9. COMMON MISTAKES

- Only checking application health.
- Ignoring data dependency health.
- Using long DNS TTLs.
- Not testing full end-to-end failover.
- Assuming data is automatically available in the failover region.
- Overlooking session affinity.
- Not monitoring external dependencies.
- Relying on a single health signal.
- Underestimating cross-region costs and latency.

SENIOR ENGINEER INSIGHT

A successful failover is not just about routing traffic — it requires all dependencies to be healthy and accessible.
- Design and test multi-region architectures as a complete system, not just individual components.

REAL-WORLD TIP

Regularly run game days to simulate regional failures.
- Validate both application and data service recovery.
- Use observability (metrics, logs, tracing) to confirm full health across regions.

PRODUCTION CHECKLIST

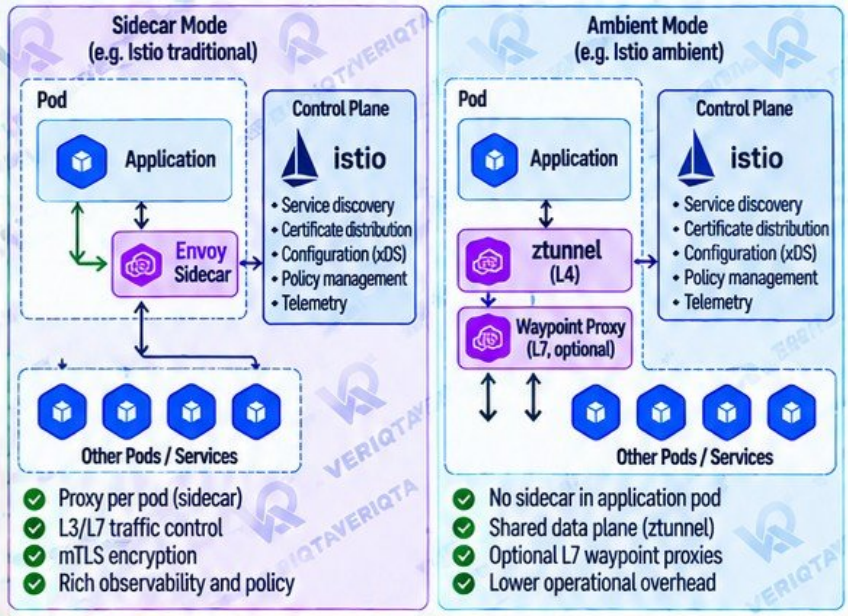
- Verify DNS health checks and routing policies.
- Confirm data replication and availability.
- Test failover and rollback procedures.
- Monitor application and all dependencies.
- Review and optimise DNS TTL and routing configurations.

Service Mesh: mTLS, Sidecars and Operational Cost

Explore sidecar and ambient datapaths, identity, traffic policy, Envoy connection pools, certificate distribution, control-plane dependencies, and proxy resource overhead.

Production Case: A mesh configuration change disrupts east-west traffic across multiple services.

1. SERVICE MESH ARCHITECTURE (SIDECAR vs AMBIENT)



- Proxy per pod (sidecar)
- L3/L7 traffic control
- mTLS encryption
- Rich observability and policy

- No sidecar in application pod
- Shared data plane (ztunnel)
- Optional L7 waypoint proxies
- Lower operational overhead

2. KEY COMPONENTS AND THEIR ROLE

Component	Purpose	Key Details
Data Plane (Envoy Proxy)	Handles inbound and outbound traffic for services	mTLS, traffic policy, retries, circuit breaking, telemetry
Control Plane (Istiod)	Distributes configuration, certificates, and policies	xDS (LDS, CDS, RDS, EDS), certificate rotation, service discovery
Identity & mTLS	Authenticates services and encrypts traffic	SPIFFE identities, automatic cert distribution and rotation
Traffic Policy	Controls how traffic flows between services	Routing, retries, timeouts, load balancing, fault injection, authorization
Envoy Connection Pools	Manages upstream connections efficiently	Limits, max connections, keep-alive, circuit breakers, outlier detection
Observability	Provides metrics, logs and traces	Access logs, metrics (Envoy / Prometheus), distributed tracing
Operational Cost	Resource overhead and complexity	Extra CPU/memory per proxy, control plane high availability, upgrades

3. mTLS AND SERVICE IDENTITY FLOW

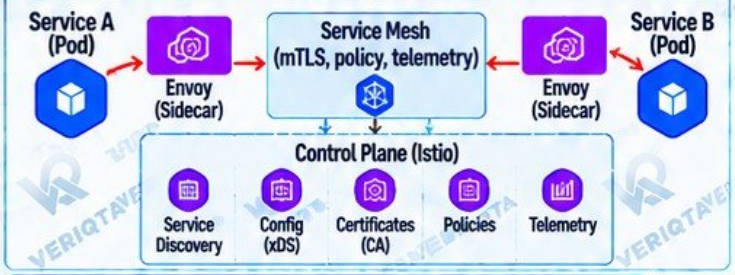


1. Both services have SPIFFE identities.
2. Envoy requests certificates from control plane.
3. mTLS encrypts and authenticates traffic.
4. Policies are enforced based on identity, namespace, service account, etc.

4. TRAFFIC POLICY EXAMPLES

Policy Type	Example	Use Case
Traffic Routing	Route 10% to v2	Canary releases
Retry Policy	Retry 3 times with exponential backoff	Handle transient failures
Timeouts	2s request timeout	Prevent hanging requests
Circuit Breaking	Max 100 connections	Protect downstream services
Fault Injection	Introduce 5% latency	Resilience testing
Authorization	Allow only specific services	Zero-trust security
Traffic Shaping	Rate limiting	Prevent abuse and control load

5. SERVICE MESH TRAFFIC FLOW (EAST-WEST)



6. COMMON FAILURE CAUSES (EAST-WEST TRAFFIC DISRUPTION)

Cause	Explanation
Incorrect mTLS policy	mTLS enforced but missing or invalid certificates.
Misconfigured authorization	Traffic blocked by new policy or RBAC rules.
Control plane issues	Control plane unavailable or unable to push config.
Proxy version mismatch	Incompatible Envoy versions after upgrade.
Destination rule misconfiguration	Wrong subsets, ports or TLS settings.
Network policy conflict	Kubernetes NetworkPolicy blocking mesh traffic.
Resource exhaustion	High CPU/memory usage on proxies.
Certificate distribution failure	Proxies cannot obtain or rotate certificates.
DNS or service discovery issues	Services not found or resolved incorrectly.

7. USEFUL DIAGNOSTIC COMMANDS

```
kubectl get pods -n istio-system          # Check control plane and proxies
istioctl proxy-status                     # Proxy sync status
istioctl pc clusters <pod>                # View Envoy clusters
istioctl pc listeners <pod>               # View Envoy listeners
istioctl pc routes <pod>                  # View routing configuration
istioctl pc endpoints <pod>               # Check service endpoints
istioctl authn tls-check <pod>            # Verify mTLS status
kubectl logs <pod> -c istio-proxy         # Check proxy logs
curl -v --cacert <cert> https://service:port # Test mTLS connectivity
```

8. BEST PRACTICES

- Use automated certificate management.
- Design clear and consistent traffic policies.
- Monitor proxy resource usage (CPU/memory).
- Keep control plane highly available.
- Test changes in a staging environment.
- Use progressive rollouts for policy changes.
- Implement proper observability (metrics, logs, traces).
- Regularly review and clean up unused configurations.
- Plan for upgrades and version compatibility.

9. COMMON MISTAKES

- Enforcing mTLS without testing.
- Overly restrictive authorization policies.
- Ignoring control plane health.
- Underestimating proxy resource overhead.
- Not monitoring Envoy metrics and logs.
- Making multiple configuration changes at once.
- Skipping staging environment tests.
- Ignoring certificate rotation and expiry.
- Assuming service mesh solves all security issues.

SENIOR ENGINEER INSIGHT

- A service mesh adds powerful capabilities, but it also introduces new operational complexity and cost.
- Always understand the data plane and control plane dependencies.
- Monitor proxy health, resource usage and certificate status.

REAL-WORLD TIP

- Use mesh visualization tools (e.g. Kiali) to understand traffic flows and quickly identify issues.
- Set up alerts for control plane, certificate expiry and proxy resource usage.
- Always have a rollback plan for mesh configuration changes.

PRODUCTION CHECKLIST

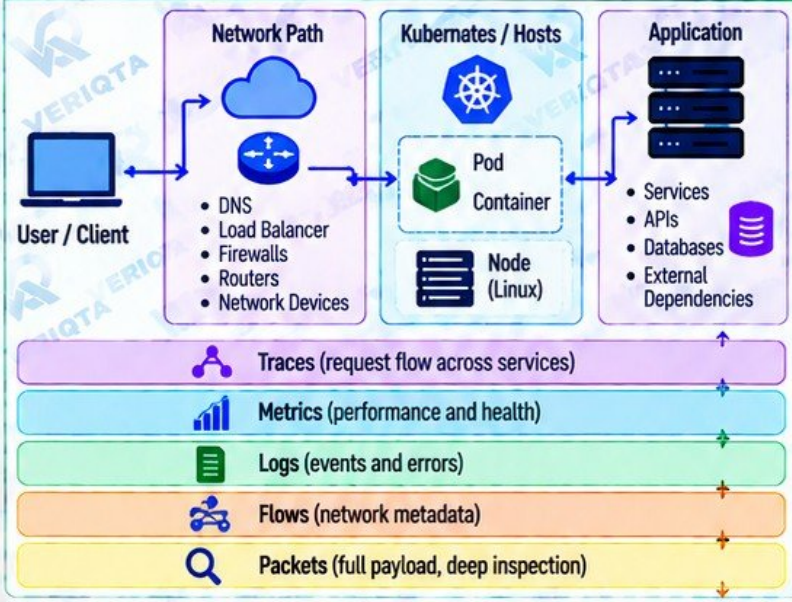
- Verify control plane health and redundancy.
- Confirm certificates are valid and rotating.
- Validate traffic policies in staging before production.
- Monitor proxy CPU, memory and connection pools.
- Ensure observability (metrics, logs, traces) is enabled.
- Have a tested rollback plan.

Network Observability: Packets, Flows and eBPF

Correlate packet captures, flow logs, socket telemetry, eBPF events, TCP retransmissions, DNS latency, and distributed traces. Understand sampling limitations and instrumentation overhead.

Production Case: Application metrics report timeouts, but infrastructure dashboards show no obvious fault.

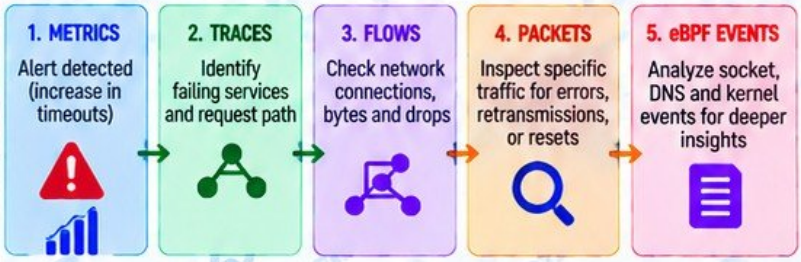
1. NETWORK OBSERVABILITY LAYERS (END-TO-END VIEW)



2. OBSERVABILITY DATA SOURCES

Data Source	What It Shows	Strengths	Limitations
Packet Capture (tcpdump, Wireshark)	Full packet content	Deep inspection, protocol details	High storage, performance overhead
Flow Logs (VPC, NetFlow, sFlow, eBPF)	Connection metadata (5-tuple, bytes)	Low overhead, wide visibility	No payload, limited detail
Socket Telemetry (eBPF)	Socket events, latency, retransmissions	Application-level network insights	Requires instrumentation and kernel support
eBPF Events (Cilium, Pixie, BCC)	Real-time network, DNS, TCP and kernel events	High visibility, low overhead	May require privileged access, learning curve
Distributed Traces (OpenTelemetry)	End-to-end request flow	Correlates across services	Sampling may miss requests, adds overhead
Metrics (Prometheus)	Latency, errors, traffic volume	Real-time alerting	May not show root cause
DNS Logs	DNS queries and response times	Identify DNS latency and failures	May not include application context

3. CORRELATING DATA FOR ROOT CAUSE ANALYSIS



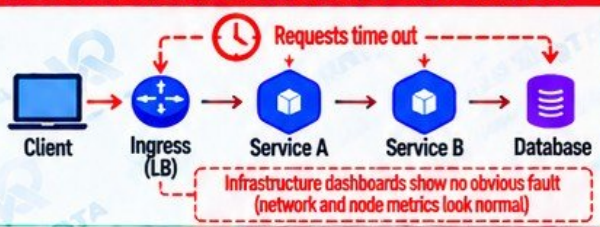
4. COMMON NETWORK OBSERVABILITY SIGNALS

Signal	What to Look For	Possible Issue
TCP Retransmissions	High retransmission rate	Packet loss, congestion, or MTU issues
DNS Latency	Slow DNS queries or failures	DNS server issues, network path problems
Connection Errors	SYN, RST, timeout errors	Service unavailable, firewall drops, routing issues
High Latency	Increased round-trip time (RTT)	Network congestion, suboptimal routing
Flow Log Drops	Rejected or dropped flows	Security group, NACL, firewall rules
eBPF Socket Events	Connection resets, retries, slow handshakes	Application or network instability
Trace Gaps	Missing spans	Sampling, instrumentation issues, or failed requests

5. EXAMPLE OBSERVABILITY WORKFLOW



6. COMMON FAILURE CAUSE: METRICS SHOW TIMEOUTS



7. USEFUL DIAGNOSTIC COMMANDS

```
tcpdump -i any port 443                    # Capture packets
wireshark                                  # Analyze packet capture
kubectl exec -it <pod> -- curl -v <service>  # Test connectivity
dig <domain>                               # Check DNS resolution
nslookup <domain>                          # Alternative DNS check
ss -tuna                                   # Check socket connections
netstat -s                                 # TCP statistics
bpftool prog show                          # List eBPF programs
cilium monitor                             # Monitor eBPF network events
kubectl logs <pod>                         # Check application logs
curl -w "@curl-format.txt" <url>           # Detailed request metrics
```

8. BEST PRACTICES

- Use multiple data sources (metrics, logs, traces, flows, packets).
- Correlate data across layers and timeframes.
- Understand sampling limitations in traces.
- Minimize instrumentation overhead.
- Monitor DNS latency and TCP retransmissions.
- Use eBPF for low-overhead, deep visibility.
- Set up meaningful alerts and dashboards.
- Document and automate diagnostic workflows.
- Regularly test observability in failure scenarios.

9. COMMON MISTAKES

- Relying only on metrics.
- Ignoring DNS latency and failures.
- Not correlating traces with network data.
- Using high sampling rates in production.
- Overlooking TCP retransmissions.
- Not accounting for instrumentation overhead.
- Assuming infrastructure dashboards show all issues.
- Not testing observability tools regularly.
- Ignoring application-level context.

SENIOR ENGINEER INSIGHT

- Metrics tell you there is a problem, but not the why.
- Packet, flow and eBPF data reveal the real network behavior.
- Correlating multiple signals is essential for accurate root cause.

REAL-WORLD TIP

Create a runbook for network troubleshooting.
- Use eBPF tools like Cilium, Pixie or BCC in production.
- Combine traces, metrics, logs and packet captures.
- Test observability during game days and failure drills.

PRODUCTION CHECKLIST

- Metrics, logs, traces and flow logs enabled.
- eBPF observability tools deployed.
- DNS and TCP monitoring configured.
- Dashboards and alerts set up.
- Diagnostic runbooks documented and tested.

Network Performance: Tail Latency and Queueing

Investigate throughput versus latency, queue buildup, bufferbloat, bandwidth-delay product, NIC offloads, CPU affinity, connection pooling, and saturation signals.

Production Case: Throughput improves after tuning, but P99 latency becomes unacceptable.

1. THROUGHPUT VS LATENCY (KEY CONCEPTS)

Throughput

Amount of data successfully transferred in a given time.



- **Measured in:** bps (Kbps, Mbps, Gbps)
- Higher throughput = more capacity
- **Affected by:** bandwidth, CPU, NIC, congestion, application efficiency

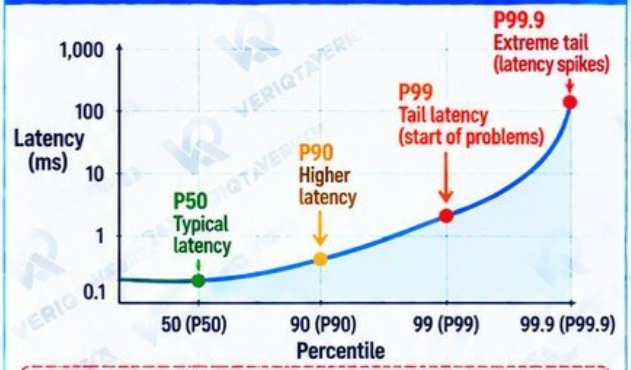
Latency

Time taken for a packet or request to travel from source to destination.



- **Measured in:** ms (milliseconds)
- **Includes:** network, processing, queuing, serialization, transmission
- Lower latency = faster response

2. LATENCY DISTRIBUTION AND TAIL LATENCY



Tail latency matters because a small number of very slow requests can significantly impact user experience even when average latency looks good.

3. QUEUEING, BUFFERBLOAT AND BANDWIDTH-DELAY PRODUCT

Queueing



- Packets wait in queues when downstream resources are busy.
- Causes increased latency, especially under load.

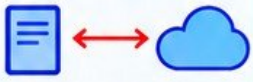
Bufferbloat



- Excessive buffering in network devices.
- Keeps queues full, increasing latency even when bandwidth is available.
- Common in poorly configured routers and NIC buffers.

Bandwidth-Delay Product (BDP)

BDP = Bandwidth × RTT

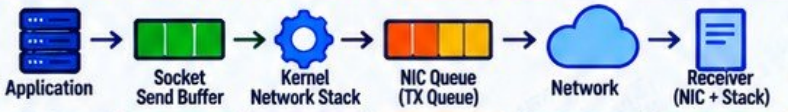


- Represents the amount of data "in flight" on the network.
- Helps size buffers, windows and connection parameters for optimal performance.

4. FACTORS AFFECTING NETWORK PERFORMANCE

Factor	Impact on Performance	Key Considerations
Network Bandwidth	Limits maximum throughput	Link speed, shared capacity, contention
Round Trip Time (RTT)	Affects latency and BDP	Geographic distance, routing path
Queueing and Buffers	Increases latency when overloaded	Buffer sizes, queue management (AQM)
NIC Offloads	Can improve throughput and CPU usage	TSO, GRO, LRO, checksum offload (may affect latency and observability)
CPU Affinity	Reduces context switching and jitter	Pin network interrupts and application threads
Connection Pooling	Reduces handshake overhead and latency	Pool size, idle timeouts, max connections
Saturation Signals	Indicates resource bottlenecks	High CPU, high queue depth, packet drops, retransmissions.

5. NETWORK DATA FLOW AND QUEUEING POINTS



Application	Socket Send Buffer	Kernel Network Stack	NIC Queue (TX Queue)	Network	Receiver (NIC + Stack)
Generates data (requests/ responses)	Data queued in socket buffers	Processed by TCP/IP stack	Packets queued in NIC transmit ring	Traverses network (routers, links)	Received and processed by destination

6. COMMON PERFORMANCE ISSUES

Issue	Symptoms	Root Cause	Result
High tail latency	P99 latency spikes	Queue buildup, GC, lock contention	Slow user requests
Bufferbloat	High latency under load	Excessive buffering	Increased tail latency
CPU saturation	High CPU usage	Insufficient CPU, poor affinity	Packet drops, latency
NIC queue overflow	TX/RX drops	High traffic, small queue size	Retransmissions, latency
TCP retransmissions	Increased retransmits	Packet loss, congestion	Throughput and latency degradation
Connection pool exhaustion	Slow or failed requests	Too few connections	Increased latency, timeouts

7. DIAGNOSTIC COMMANDS

```
1  iperf3 -c <server> -t 30   # Measure throughput
2  ping <host>                # Check latency and packet loss
3  mtr <host>                 # Identify network path issues
4  tc -s qdisc                # View queueing statistics
5  ss -s                      # Check socket statistics
6  ethtool -S <iface>         # NIC statistics (drops, errors)
7  sar -n DEV                 # Network interface metrics
8  perf top                   # CPU hotspots
9  tcpdump -i <iface>         # Capture packets
10 nstat                      # TCP statistics
                                (retransmissions, etc.)
```

8. BEST PRACTICES

- Monitor and optimize P50, P95 and P99 latency.
- Right-size buffers using bandwidth-delay product (BDP).
- Use modern queue management (e.g. fq_codel, CAKE).
- Tune NIC offloads and interrupt coalescing.
- Use CPU affinity for network and application workloads.
- Implement and tune connection pooling.
- Monitor saturation signals (CPU, queues, drops, retransmits).
- Test under realistic load and network conditions.

9. COMMON MISTAKES

- Focusing only on average latency (P50).
- Ignoring tail latency (P99, P99.9).
- Using excessively large buffers.
- Disabling useful NIC offloads without testing.
- Not tuning TCP parameters.
- Underestimating the impact of CPU contention.
- Using too few or too many connections.
- Not monitoring queue depth and drops.
- Assuming throughput improvement automatically means better latency.

10. PRODUCTION CHECKLIST

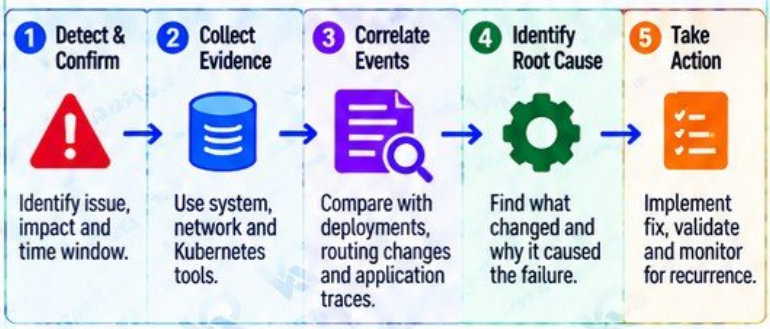
- [ ] Establish latency and throughput baselines (P50, P95, P99).
- [ ] Monitor queue depths, drops and retransmissions.
- [ ] Verify NIC and system tuning (offloads, affinity).
- [ ] Check connection pool configuration.
- [ ] Alert on tail latency and saturation signals.
- [ ] Test changes under realistic load.
- [ ] Document optimal settings for your environment.

Network Incident Forensics: Evidence Before Action

Build an investigation workflow using ss, ip, dig, tcpdump, conntrack, nstat, and Kubernetes diagnostics. Correlate packet evidence with deployments, routing changes, and application traces.

Production Case: Intermittent network failures occur only during peak production traffic.

1. INCIDENT INVESTIGATION WORKFLOW (OVERVIEW)



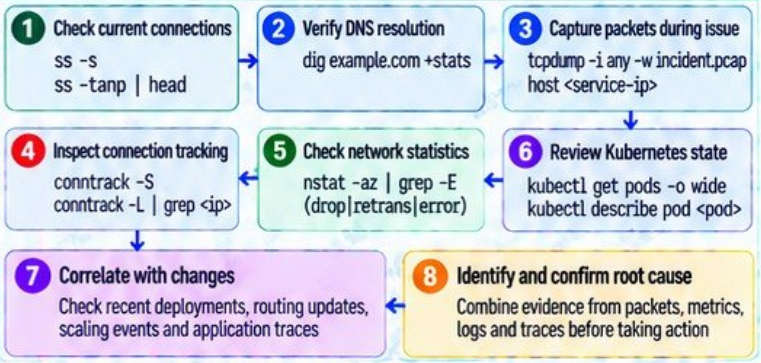
2. KEY COMMANDS FOR NETWORK FORENSICS

Command	Purpose	Example	What to Look For
ss	View sockets and connections	ss -tulpn	Established, SYN_RECV, HIGH connection count
ip	Inspect network configuration and routes	ip addr ip route	IP changes, missing routes, incorrect interfaces
dig	DNS lookup and latency	dig example.com +stats	High latency, SERVFAIL, wrong IPs
tcpdump	Capture and analyze packets	tcpdump -i any host 10.0.0.1	Retransmissions, drops, unusual traffic
conntrack	View connection tracking table	conntrack -L	High entry count, FAILED states, timeouts
nstat	Network protocol statistics	nstat -az	Errors, retransmissions, packet drops
kubectl	Kubernetes diagnostics	kubectl get pods kubectl describe pod	Pod status, events, network policies, CNI issues

3. DATA SOURCES TO CORRELATE



4. INVESTIGATION EXAMPLE: STEP-BY-STEP



5. EXAMPLE PACKET FLOW ANALYSIS



Normal Traffic (Healthy)

```
$ tcpdump -n -i any host 10.0.0.5
...
10:00:01.123 IP 10.0.0.10.54321 > 10.0.0.5.80: Flags [S]
10:00:01.124 IP 10.0.0.5.80 > 10.0.0.10.54321: Flags [S.]
10:00:01.125 IP 10.0.0.10.54321 > 10.0.0.5.80: Flags [.]
```

Problematic Traffic (During Peak)

```
$ tcpdump -n -i any host 10.0.0.5
...
10:15:20.001 IP 10.0.0.10.54321 > 10.0.0.5.80: Flags [S]
10:15:20.501 IP 10.0.0.10.54321 > 10.0.0.5.80: Flags [S]
10:15:21.001 IP 10.0.0.10.54321 > 10.0.0.5.80: Flags [S]
(retransmissions, no response)
```

6. COMMON FINDINGS IN INCIDENTS

Finding	Symptoms	Possible Cause	Next Step
High SYN_RECV connections	Slow or failed connections	Server overload, SYN flood, backlog full	Check server resources, SYN backlog, scaling
DNS latency	Intermittent lookup failures	DNS server issues, network congestion	Test alternative DNS, check network path
Packet loss or retransmissions	High latency, timeouts	Network congestion, queue buildup, faulty link	Analyze tcpdump, nstat, check bandwidth and queues
Conntrack table full	New connections failing	High connection churn, insufficient conntrack size	Increase conntrack limits, optimize connection reuse
Kubernetes CNI issues	Pods cannot communicate	CNI misconfiguration, network policy, node issue	Check CNI logs, policies, node health
Routing changes	Traffic to wrong destination	Incorrect route, BGP/DNS change, misconfiguration	Verify routes, rollback change

7. BEST PRACTICES

- Collect evidence before making changes.
- Use multiple data sources (packets, logs, metrics, traces).
- Capture data during the issue, not after.
- Correlate with deployments, scaling events and routing changes.
- Look at both north-south and east-west traffic.
- Check for DNS, load balancer and service mesh involvement.
- Consider sampling limitations in logs and metrics.
- Validate findings in a test environment when possible.
- Document timeline and findings for post-incident review.

8. COMMON MISTAKES

- Making changes before collecting evidence.
- Relying only on metrics or only on logs.
- Ignoring DNS and name resolution issues.
- Not capturing packets during the actual incident.
- Overlooking connection tracking and kernel statistics.
- Assuming the issue is at the application layer.
- Not correlating with deployments or infrastructure changes.
- Stopping data collection too early.
- Drawing conclusions from incomplete data.

SENIOR ENGINEER INSIGHT

Always capture a baseline when the system is healthy.
- Network issues are often timing-related and only appear under load.
- The combination of packet captures, system metrics and application traces provides the most reliable evidence.

REAL-WORLD TIP

Use time synchronization (NTP) across all systems.
- Capture short packet samples regularly in high-traffic environments.
- Keep conntrack, nstat and key metrics in your monitoring dashboards.
- Build an incident runbook with commands and expected outputs.

PRODUCTION CHECKLIST

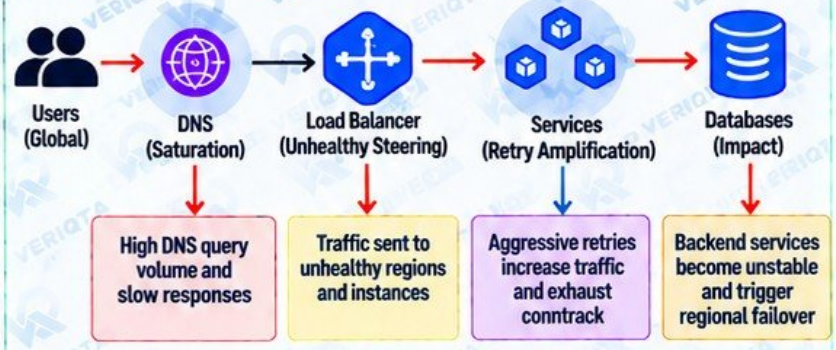
- Verify monitoring and alerting is in place.
- Ensure packet capture tools are available.
- Confirm log retention and trace sampling.
- Test incident runbook in a staging environment.
- Review and improve based on post-incident findings.

Principal Engineer Case Study: Surviving a Network Cascade

A combined incident involving DNS saturation, retry amplification, conntrack exhaustion, unhealthy traffic steering, and regional failover. Diagnose the causal chain, prioritize mitigations, protect unaffected regions, and design long-term resilience.

Final Challenge: Defend an architecture, incident response, and verification plan before a production readiness review.

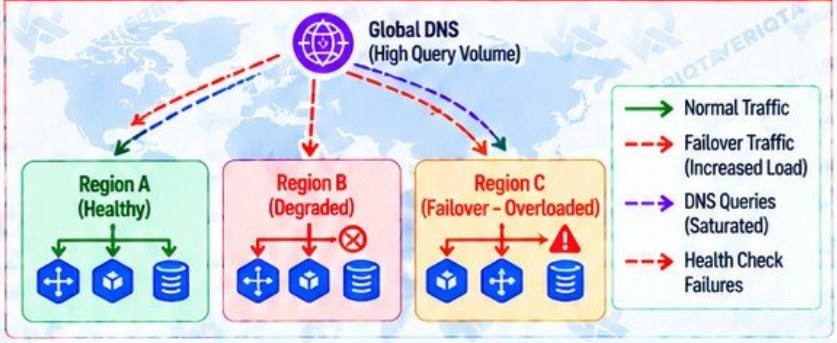
1. INCIDENT SCENARIO OVERVIEW



2. INCIDENT TIMELINE AND CAUSAL CHAIN

1. **DNS Saturation** — Sudden spike in DNS queries increases latency and causes timeouts for some resolvers.
2. **Retry Amplification** — Clients and services retry aggressively, multiplying traffic across the system.
3. **Conntrack Exhaustion** — High connection rate exhausts conntrack table on nodes and load balancers, dropping new connections.
4. **Unhealthy Traffic Steering** — Health checks become unreliable, sending traffic to unhealthy instances and regions.
5. **Regional Failover** — Traffic shifts to another region, increasing load and causing further degradation.
6. **Full Impact** — Multiple services become unavailable, affecting users globally.

3. MULTI-REGION ARCHITECTURE DURING INCIDENT



4. KEY TECHNICAL AREAS INVOLVED

DNS	Query saturation, resolver latency, TTL, caching.
Retries	Client and service retry behavior, exponential backoff.
Conntrack	Conntrack table limits, connection tracking timeouts.
Load Balancing	Health checks, traffic steering, failure detection.
Kubernetes	Readiness, liveness, service endpoints, kube-proxy.
Database	Connection limits, read replicas, failover behavior.
Regional Failover	DNS failover, traffic shift, cross-region dependencies.
Observability	Metrics, logs, traces, packet captures, flow logs.

5. INVESTIGATION WORKFLOW

1. **Detect and Assess** — Identify symptoms and blast radius.
2. **Collect Evidence** — Gather DNS logs, packet captures, conntrack, metrics, and traces.
3. **Correlate Events** — Build timeline across DNS, network, kube, and application logs.
4. **Find Root Cause** — Identify the primary trigger and contributing factors.
5. **Prioritize Mitigations** — Stabilize service, protect healthy regions, reduce load.
6. **Validate and Learn** — Confirm recovery, verify fixes, and document lessons learned.

6. EXAMPLE COMMANDS FOR INVESTIGATION

```
dig example.com +stats          # Check DNS latency and response
nslookup example.com            # Verify DNS resolution
ss -s                           # Socket and connection summary
ss -tanp                        # Active connections and states
sudo conntrack -S               # Conntrack table statistics
sudo conntrack -L               # View current conntrack entries
tcpdump -ni any                 # Capture and analyze packets
nstat -az                       # Network protocol statistics
kubectl get pods -A             # Check pod status across cluster
kubectl get endpoints -A        # Verify service endpoints
ip route show                   # Check routing table
curl -w "%{time_total}\n"       # Test request latency
```

7. MITIGATION STRATEGY (DURING INCIDENT)

- Reduce DNS query load and enable caching.
- Implement and tune client retry and backoff settings.
- Increase conntrack limits and optimize timeouts.
- Remove unhealthy instances from load balancers.
- Protect unaffected regions from extra failover load.
- Scale critical services and databases.
- Use traffic shaping and rate limiting.
- Communicate status and manage user impact.

8. LONG-TERM RESILIENCE DESIGN

- Scalable and redundant DNS architecture.
- Circuit breakers and controlled retries.
- Proper conntrack and kernel tuning.
- Robust health checks and traffic steering.
- Multi-region active-active or active-passive design.
- Dependency-aware failover and data consistency.
- Comprehensive observability and alerting.
- Regular game days and failure testing.

9. FINAL REVIEW PRESENTATION

- **Architecture Design** — Defend a resilient, scalable architecture.
- **Incident Response Plan** — Explain detection, investigation and mitigation.
- **Verification Plan** — Show how to validate recovery and prevent recurrence.
- **Risk and Trade-offs** — Discuss costs, complexity and operational impact.
- **Lessons Learned** — Key takeaways and improvements.

SENIOR ENGINEER INSIGHT

- Network incidents are rarely caused by a single issue.
- Evidence-based investigation prevents wrong fixes.
- Protect healthy regions and users first.

REAL-WORLD TIP

- Build and test runbooks for network cascades.
- Simulate failures with game days.
- Use detailed metrics, logs and traces for verification.

PRODUCTION CHECKLIST

- DNS, retries, conntrack, health checks and failover tested.
- Monitoring and alerting in place.
- Runbooks documented and regularly practiced.